



(12)发明专利

(10)授权公告号 CN 107678790 B

(45)授权公告日 2020.05.08

(21)申请号 201610617253.2

G06F 16/25(2019.01)

(22)申请日 2016.07.29

(56)对比文件

(65)同一申请的已公布的文献号

申请公布号 CN 107678790 A

CN 104504143 A, 2015.04.08,

CN 104199831 A, 2014.12.10,

CN 105404690 A, 2016.03.16,

CN 101192239 A, 2008.06.04,

CN 105786808 A, 2016.07.20,

CN 103870340 A, 2014.06.18,

CN 103970602 A, 2014.08.06,

WO 2016088281 A1, 2016.06.09,

(43)申请公布日 2018.02.09

(73)专利权人 华为技术有限公司

地址 518129 广东省深圳市龙岗区坂田华为总部办公楼

(72)发明人 史云龙 方丰斌

审查员 刘迪

(74)专利代理机构 北京三高永信知识产权代理有限公司 11138

代理人 罗振安

(51)Int.Cl.

G06F 8/61(2018.01)

G06F 9/54(2006.01)

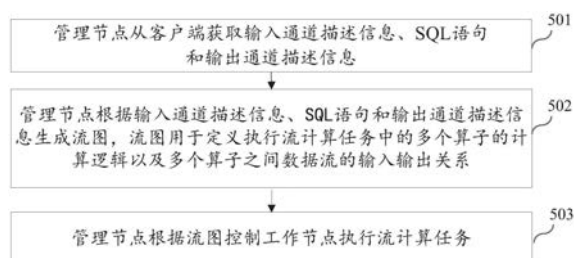
权利要求书3页 说明书21页 附图12页

(54)发明名称

流计算方法、装置及系统

(57)摘要

本发明公开了一种流计算方法、装置及系统,属于大数据计算领域。所述方法包括:管理节点获取输入通道描述信息、SQL语句和输出通道描述信息;根据输入通道描述信息、SQL语句和输出通道描述信息生成流图;根据流图控制工作节点执行流计算任务。本发明一定程度上解决了目前的流计算系统提供的基本算子的功能粒度太细,导致构建流图的复杂度较高且性能较差的问题;由于SQL是较为常见的数据库管理语言,由用户利用SQL语言的编程语言特性,采用SQL语句对流图的处理逻辑进行定义,由管理节点按照SQL语句所定义的处理逻辑动态生成流图,从而提高构建流图的易用性和流图的整体计算性能。



1. 一种流计算方法,其特征在于,应用于包括管理节点和工作节点的流计算系统中,所述方法包括:

所述管理节点从客户端获取输入通道描述信息、结构化查询语言SQL语句和输出通道描述信息,所述SQL语句包括若干条SQL规则,每条SQL规则包括至少一条SQL子语句;

所述管理节点根据所述输入通道描述信息、所述若干条SQL规则和所述输出通道描述信息生成第一流图,所述第一流图包括若干个逻辑节点;

所述管理节点将所述第一流图中的各个逻辑节点进行划分,以得到若干个逻辑节点组;在预设算子库中选择每个所述逻辑节点组对应的公共算子,并根据选择出的所述公共算子生成第二流图;所述第二流图中的每个算子用于实现所述算子对应的逻辑节点组中的一个或多个逻辑节点的功能;所述管理节点根据所述第二流图控制所述工作节点执行所述流计算任务;

其中,所述输入通道描述信息用于定义输入通道,所述输入通道是用于将来自数据生产系统的数据流输入流图的逻辑通道;所述输出通道描述信息用于定义输出通道,所述输出通道是用于将流图的输出数据流输出至数据消费系统的逻辑通道。

2. 根据权利要求1所述的方法,其特征在于,所述第一流图包括通过有向边相连的源逻辑节点、中间逻辑节点和目标逻辑节点;所述第二流图包括通过有向边相连的源算子、中间算子和目标算子。

3. 根据权利要求2所述的方法,其特征在于,所述管理节点根据所述输入通道描述信息、所述若干条SQL规则和所述输出通道描述信息生成第一流图,包括:

所述管理节点根据所述输入通道描述信息生成所述第一流图中的所述源逻辑节点,所述源逻辑节点用于接收来自所述数据生产系统的输入数据流;

所述管理节点根据每条所述SQL规则中的选择子语句生成所述第一流图中的所述中间逻辑节点,所述中间逻辑节点用于指示对所述输入数据流进行计算时的计算逻辑,每个中间逻辑节点对应一条SQL规则;

所述管理节点根据所述输出通道描述信息生成所述第一流图中的目标逻辑节点,所述目标逻辑节点用于向所述数据消费系统发送输出数据流;

所述管理节点根据每条所述SQL规则中的输入子语句和/或输出子语句,生成所述源逻辑节点、所述中间逻辑节点以及所述目标逻辑节点之间的有向边。

4. 根据权利要求3所述的方法,其特征在于,所述预设算子库包括:公共源算子、公共中间算子和公共目标算子;

所述在预设算子库中选择每个所述逻辑节点组对应的公共算子,并根据选择出的所述公共算子生成第二流图包括:

编译所述公共源算子以得到所述第二流图中的源算子;

在所述预设算子库中为每个包括所述中间逻辑节点的所述逻辑节点组选择出至少一个公共中间算子,编译选择出的所述公共中间算子以得到所述第二流图中的中间算子;

编译所述公共目标算子以得到所述第二流图中的目标算子;

根据所述源逻辑节点、所述中间逻辑节点与所述目标逻辑节点之间的有向边,生成所述第二流图中的各个算子之间的有向边。

5. 根据权利要求4所述的方法,其特征在于,所述方法还包括:

所述管理节点接收来自所述客户端的第一修改信息,所述第一修改信息是对所述SQL规则进行修改的信息;

所述管理节点根据所述第一修改信息对所述第二流图中对应的所述中间算子进行增加、修改或删除。

6. 根据权利要求4或5所述的方法,其特征在于,所述方法还包括:

所述管理节点接收来自所述客户端的第二修改信息,所述第二修改信息是对所述输入通道描述信息进行修改的信息;根据所述第二修改信息对所述第二流图中的所述源算子进行增加、修改或删除;

和/或,

所述管理节点接收来自所述客户端的第三修改信息,所述第三修改信息是对所述输出通道描述信息进行修改的信息;根据所述第三修改信息对所述第二流图中的所述目标算子进行增加、修改或删除。

7. 根据权利要求1至4任一所述的方法,其特征在于,所述流计算系统包括多个工作节点,所述管理节点根据所述第二流图控制所述工作节点执行所述流计算任务,包括:

所述管理节点将所述第二流图中的各个所述算子调度至所述流计算系统中的至少一个工作节点中,所述至少一个工作节点用于执行所述算子;

所述管理节点根据每个所述算子的输出数据流,生成与所述算子对应的订阅发布信息,向所述算子配置所述订阅发布信息;

所述管理节点根据每个所述算子的输入数据流,生成与所述算子对应的输入流定义信息,向所述算子配置所述输入流定义信息;

其中,所述订阅发布信息用于指示与当前算子对应的输出数据流的发送方式;所述输入流定义信息用于指示与当前算子对应的输入数据流的接收方式。

8. 一种流计算装置,其特征在于,所述装置包括:

获取单元,用于从客户端获取输入通道描述信息、结构化查询语言SQL语句和输出通道描述信息,所述SQL语句包括若干条SQL规则,每条SQL规则包括至少一条SQL子语句;

生成单元,用于根据所述输入通道描述信息、所述若干条SQL规则和所述输出通道描述信息生成第一流图,所述第一流图包括若干个逻辑节点;将所述第一流图中的各个逻辑节点进行划分,以得到若干个逻辑节点组;在预设算子库中选择每个所述逻辑节点组对应的公共算子,并根据选择出的所述公共算子生成第二流图;所述第二流图中的每个算子用于实现所述算子对应的逻辑节点组中的一个或多个逻辑节点的功能;

执行单元,根据所述第二流图控制流计算系统中工作节点执行所述流计算任务;

其中,所述输入通道描述信息用于定义输入通道,所述输入通道是用于将来自数据生产系统的数据流输入流图的逻辑通道;所述输出通道描述信息用于定义输出通道,所述输出通道是用于将流图的输出数据流输出至数据消费系统的逻辑通道。

9. 根据权利要求8所述的装置,其特征在于,所述第一流图包括通过有向边相连的源逻辑节点、中间逻辑节点和目标逻辑节点;所述第二流图包括通过有向边相连的源算子、中间算子和目标算子。

10. 根据权利要求9所述的装置,其特征在于,

所述生成单元,具体用于根据所述输入通道描述信息生成所述第一流图中的所述源逻

辑节点,所述源逻辑节点用于接收来自所述数据生产系统的输入数据流;根据每条所述SQL规则中的选择子语句生成所述第一流图中的所述中间逻辑节点,所述中间逻辑节点用于指示对所述输入数据流进行计算时的计算逻辑,每个中间逻辑节点对应一条SQL规则;根据所述输出通道描述信息生成所述第一流图中的目标逻辑节点,所述目标逻辑节点用于向所述数据消费系统发送输出数据流;根据每条所述SQL规则中的输入子语句和/或输出子语句,生成所述源逻辑节点、所述中间逻辑节点、以及所述目标逻辑节点之间的有向边。

11. 根据权利要求10所述的装置,其特征在于,所述预设算子库包括:公共源算子、公共中间算子和公共目标算子;

所述生成单元,具体用于编译所述公共源算子以得到所述第二流图中的源算子;在所述预设算子库中为每个包括所述中间逻辑节点的所述逻辑节点组选择出至少一个公共中间算子,编译选择出的所述公共中间算子以得到所述第二流图中的中间算子;编译所述公共目标算子以得到所述第二流图中的目标算子;根据所述源逻辑节点、所述中间逻辑节点与所述目标逻辑节点之间的有向边,生成所述第二流图中的各个算子之间的有向边。

12. 根据权利要求11所述的装置,其特征在于,所述装置还包括:修改单元;

所述获取单元,还用于接收来自所述客户端的第一修改信息,所述第一修改信息是对所述SQL规则进行修改的信息;

所述修改单元,用于根据所述第一修改信息对所述第二流图中对应的所述中间算子进行增加、修改或删除。

13. 根据权利要求11或12所述的装置,其特征在于,所述装置还包括:修改单元;

所述获取单元,还用于接收来自所述客户端的第二修改信息,所述第二修改信息是对所述输入通道描述信息进行修改的信息;所述修改单元,用于根据所述第二修改信息对所述第二流图中的所述源算子进行增加、修改或删除;

和/或,

所述获取单元,还用于接收来自所述客户端的第三修改信息,所述第三修改信息是对所述输出通道描述信息进行修改的信息;所述修改单元,用于根据所述第三修改信息对所述第二流图中的所述目标算子进行增加、修改或删除。

14. 根据权利要求8至11任一所述的装置,其特征在于,所述执行单元,用于将所述第二流图中的各个所述算子调度至所述流计算系统中的至少一个工作节点中,所述工作节点用于执行所述算子;根据每个所述算子的输出数据流,生成与所述算子对应的订阅发布信息,向所述算子配置所述订阅发布信息;根据每个所述算子的输入数据流,生成与所述算子对应的输入流定义信息,向所述算子配置所述输入流定义信息;

其中,所述订阅发布信息用于指示与当前算子对应的输出数据流的发送方式;所述输入流定义信息用于指示与当前算子对应的输入数据流的接收方式。

15. 一种流计算系统,其特征在于,所述系统包括:管理节点和至少一个工作节点;

所述管理节点包括如权利要求8至14任一所述的流计算装置。

流计算方法、装置及系统

技术领域

[0001] 本发明实施例涉及大数据计算领域,特别涉及一种流计算方法、装置及系统。

背景技术

[0002] 在诸如金融服务、传感监测和网络监控之类的应用领域中,数据流具有实时性、易失性、突发性、无序性和无限性等特征。流计算(Stream computing)系统作为一种能对实时的数据流进行计算处理的系统,得到了越来越广泛的应用。

[0003] 流计算系统中部署的流式应用(通常也成为流应用)的处理逻辑可以采用有向无环图(Directed Acyclic Graph,DAG)来表征,该DAG也称为流图。参考图1,流应用的处理逻辑采用流图100进行表征。该流图100中的每一个有向边代表一个数据流(Stream),每个节点代表一个算子(Operator),图中每个算子具有至少一条输入数据流和至少一条输出数据流。算子是流计算系统中可被调度执行计算任务的最小单元,算子也可称为执行算子。

[0004] 在将流应用部署到流计算系统时,需要由用户事先为流应用构建流图,然后流应用以流图的形式在流系统中编译和运行,以执行对数据流的处理任务。流计算系统向用户提供集成开发环境(Integrated Development Environment,IDE),该IDE提供有助于构建流图的图形用户界面,该图形用户界面中包括若干个基本算子,用户在该图形用户界面上通过拖拽基本算子的方式构建流图,并且需要为该流图配置各种运行参数。

[0005] 虽然通过拖拽基本算子的方式构建流图非常直观,但是为了便于用户构建较为复杂的流图,IDE中提供的每个基本算子的功能都被预先划分到非常细的粒度,导致构建流图的复杂度较高,且用户实际构建出的流图较为臃肿,流图的整体计算性能较差。

发明内容

[0006] 为了改善流图的整体计算性能,本发明实施例提供了一种流计算方法、装置及系统。所述技术方案如下:

[0007] 流计算系统通常采用分布式计算架构。该分布式计算架构中包括:管理节点和至少一个工作节点。用户通过客户端在管理节点中配置流图,由管理节点将流图中的各个算子调度至工作节点中运行。

[0008] 第一方面,本发明实施例提供了一种流计算方法,应用于包括管理节点和工作节点的流计算系统中,所述方法包括:管理节点从客户端获取输入通道描述信息、结构化查询语言(Structured Query Language,SQL)语句和输出通道描述信息;管理节点根据所述输入通道描述信息、所述SQL语句和所述输出通道描述信息生成流图,所述流图用于定义执行流计算任务的多个算子的计算逻辑以及各个算子之间数据流的输入输出关系;管理节点根据所述流图控制所述工作节点中的算子执行流计算任务;所述多个算子被调度到所述流计算系统的一个或多个工作节点上执行;

[0009] 其中,所述输入通道描述信息用于定义输入通道,所述输入通道是将来自数据生产系统的数据流输入所述流图的逻辑通道;所述输出通道描述信息用于定义输出通道,所

述输出通道是所述流图的输出数据流输出至数据消费系统的逻辑通道。

[0010] 本发明实施例通过由管理节点根据输入通道描述信息、SQL语句和输出通道描述信息生成可执行的流图,然后由管理节点根据流图控制工作节点执行流计算;一定程度上解决了目前的流计算系统通过IDE提供的基本算子来构建流图时,每个基本算子的功能被划分为非常细的粒度,导致构建流图的复杂度较高,且生成的流图的整体计算性能较差的问题;SQL是较为常见的数据库管理语言,流计算系统支持SQL语句来构建流图可以提高系统的易用性,提升用户体验,另一方面,由用户利用SQL语言的编程语言特性,采用SQL语句对流图的处理逻辑进行定义,由管理节点按照SQL语句所定义的处理逻辑动态生成流图,从而提高流图的整体计算性能。

[0011] 结合第一方面,在第一方面的第一种可能的实现方式中,所述SQL语句包括若干条SQL规则,每条SQL规则包括至少一条SQL子语句;

[0012] 所述管理节点根据所述输入通道描述信息、所述SQL语句和所述输出通道描述信息生成流图,具体包括:

[0013] 所述管理节点根据所述输入通道描述信息、所述若干条SQL规则和所述输出通道描述信息生成第一流图,所述第一流图包括若干个逻辑层面的节点;

[0014] 所述管理节点将所述第一流图中的各个逻辑节点进行划分,以得到若干个逻辑节点组;按照每个所述逻辑节点组在预设算子库中选择公共算子,根据选择出的所述公共算子生成第二流图;所述第二流图中的每个算子用于实现该算子对应的逻辑节点组中的一个或多个逻辑节点。

[0015] 综上所述,本实现方式提供的流计算方法,用户只需要在逻辑层面编写SQL规则,由管理节点根据SQL规则生成第一流图,第一流图包括若干个逻辑节点,然后再由管理节点通过预设算子库将第一流图中各个逻辑节点进行划分后,将每个逻辑节点组转换为第二流图中的一个算子,第二流图中的每个算子用于实现第一流图中属于同一逻辑节点组的各个逻辑节点,使得用户不需要具有流式编程思维,也不需要关心算子的划分逻辑,只需要在逻辑层面上编写SQL规则即可构建流图,由管理节点自行生成流图中的算子,从而减少了用户构建流计算应用时的代码编辑工作,降低用户构建流计算应用的复杂度。

[0016] 结合第一方面的第一种可能的实现方式,在第一方面的第二种可能的实现方式中,所述第一流图包括通过有向边相连的源逻辑节点、中间逻辑节点和目标逻辑节点,所述管理节点根据所述输入通道描述信息、所述若干条SQL规则和所述输出通道描述信息生成第一流图,具体包括:

[0017] 所述管理节点根据所述输入通道描述信息生成所述第一流图中的所述源逻辑节点,所述源逻辑节点用于接收来自所述数据生产系统的输入数据流;

[0018] 所述管理节点根据每条所述SQL规则中的选择子语句生成所述第一流图中的所述中间逻辑节点,所述中间逻辑节点用于指示对所述输入数据流进行计算时的计算逻辑,每个中间逻辑节点对应一条SQL规则;

[0019] 所述管理节点根据所述输出通道描述信息生成所述第一流图中的所述目标逻辑节点,所述目标逻辑节点用于向所述数据消费系统发送输出数据流;

[0020] 所述管理节点根据每条所述SQL规则中的输入子语句和/或输出子语句,生成所述源逻辑节点、所述中间逻辑节点与所述目标逻辑节点之间的有向边。

[0021] 综上所述,本实现方式提供的流计算方法,通过流计算系统对SQL语言中的输入子语句、选择子语句和输出子语句进行转用,实现了流计算系统支持用户使用一条SQL规则在逻辑层面上定义流图中的一个逻辑节点,利用用户熟悉的SQL语法,将定义流计算应用的难度降低,提供了易用性极高的流图定制方式。

[0022] 结合第一方面的第一种或第二种可能的实现方式,在第一方面的第三种可能的实现方式中,所述第二流图包括通过有向边相连的源算子、中间算子和目标算子,所述预设算子库包括:公共源算子、公共中间算子和公共目标算子;

[0023] 所述管理节点将所述第一流图中的各个逻辑节点进行划分,按照划分后的所述逻辑节点在预设算子库中选择公共算子,根据选择出的所述公共算子生成第二流图,包括:

[0024] 所述管理节点编译所述公共源算子以得到所述第二流图中的源算子;

[0025] 所述管理节点在所述预设算子库中为每个包括所述中间逻辑节点的所述逻辑节点组选择出至少一个公共中间算子,编译选择出的所述公共中间算子以得到所述第二流图中的中间算子;

[0026] 所述管理节点编译所述公共目标算子以得到所述第二流图中的目标算子;

[0027] 所述管理节点根据所述源逻辑节点、所述中间逻辑节点与所述目标逻辑节点之间的有向边,生成所述第二流图中的各个算子之间的有向边。

[0028] 综上所述,本实现方式提供的流计算方法,通过由管理节点将第一流图中的多个逻辑节点进行划分,将划分为同一逻辑节点组的各个逻辑节点通过同一个公共中间算子进行实现,不需要用户考虑负载均衡、并发执行等因素,由管理节点自行根据负载均衡和并发执行等因素来决策第二流图的生成,进一步地降低了用户生成第二流图时的难度,只需要用户具有通过SQL构建逻辑层面的第一流图的能力即可。

[0029] 结合第一方面的第一种或第二种或第三种可能的实现方式,在第四种可能的实现方式中,所述管理节点根据所述流图控制所述工作节点执行流计算,包括:

[0030] 所述管理节点将所述第二流图中的各个所述算子调度至流计算系统中的至少一个工作节点中,所述工作节点用于执行所述算子;

[0031] 所述管理节点根据每个所述算子的所述输出数据流,生成与所述算子对应的订阅发布信息,向所述算子配置所述订阅发布信息;

[0032] 所述管理节点根据每个所述算子的所述输入数据流,生成与所述算子对应的输入流定义信息,向所述算子配置所述输入流定义信息;

[0033] 其中,所述订阅发布信息用于指示与当前算子对应的输出数据流的发送方式;所述输入流定义信息用于指示与当前算子对应的输入数据流的接收方式。

[0034] 综上所述,本实现方式提供的流计算方法,通过设置订阅机制,将第二流图中的各个算子的输入数据流和输出数据流之间的引用关系解耦,提供了在第二流图被执行后,仍然可以动态调整第二流图中的各个算子的能力,提高了流计算应用的整体易用性和可维护性。

[0035] 结合第一方面的第一种或第二种或第三种或第四种可能的实现方式,在第五种可能的实现方式中,所述方法还包括:

[0036] 所述管理节点接收来自所述客户端的第一修改信息,所述第一修改信息是对所述SQL规则进行修改的信息;

[0037] 所述管理节点根据所述第一修改信息对所述第二流图中对应的所述中间算子进行增加、修改或删除。

[0038] 综上所述,本实现方式提供的流计算方法,通过客户端向管理节点发送第一修改信息,由管理节点根据第一修改信息对第二流图中的中间算子进行增加、修改或删除,为管理节点提供了在第二流图生成之后,仍然可以动态调整第二流图中的中间算子的能力。

[0039] 结合第一方面的第一种或第二种或第三种或第四种或第五种可能的实现方式,在第六种可能的实现方式中,所述方法还包括:

[0040] 所述管理节点接收来自所述客户端的第二修改信息,所述第二修改信息是对所述输入通道描述信息进行修改的信息;根据所述第二修改信息对所述第二流图中的所述源算子进行增加、修改或删除;

[0041] 和/或,

[0042] 所述管理节点接收来自所述客户端的第三修改信息,所述第三修改信息是对所述输出通道描述信息进行修改的信息;根据所述第三修改信息对所述第二流图中的所述目标算子进行增加、修改或删除。

[0043] 综上所述,本实现方式提供的流计算方法,通过客户端向管理节点发送第二修改信息和/或第三修改信息,由管理节点对第二流图中的源算子和/或目标算子进行增加、修改或删除,为管理节点提供了在第二流图生成之后,仍然可以动态调整第二流图中的源算子和/或目标算子的能力。

[0044] 第二方面,提供了一种流计算装置,该流计算装置包括至少一个单元,该至少一个单元用于实现上述第一方面或第一方面中任意一种可能的实现方式所提供的流计算方法。

[0045] 第三方面,提供了一种管理节点,该管理节点包括处理器和存储器;所述处理器用于存储一个或一个以上的指令,所述指令被指示为由所述处理器执行,所述处理器用于实现上述第一方面或第一方面中任意一种可能的实现方式所提供的流计算方法。

[0046] 第四方面,本发明实施例提供一种计算机可读存储介质,该计算机可读存储介质中存储有用于实现上述第一方面或第一方面中任意一种可能的实现方式所提供的流计算方法的可执行程序。

[0047] 第五方面,提供了一种流计算系统,该流计算系统包括:管理节点和至少一个工作节点,所述管理节点是如第三方面所述的管理节点。

附图说明

[0048] 为了更清楚地说明本发明实施例中的技术方案,下面将对实施例描述中所需要使用的附图作简单地介绍,显而易见地,下面描述中的附图仅仅是本发明的一些实施例,对于本领域普通技术人员来讲,在不付出创造性劳动的前提下,还可以根据这些附图获得其他的附图。

[0049] 图1是现有技术提供的流图的结构示意图;

[0050] 图2A是本发明一个实施例提供的流计算系统的结构方框图;

[0051] 图2B是本发明另一个实施例提供的流计算系统的结构方框图;

[0052] 图3A是本发明一个实施例提供的管理节点的结构方框图;

[0053] 图3B是本发明另一实施例提供的管理节点的结构方框图;

- [0054] 图4是本发明一个实施例提供的流计算过程的原理示意图；
- [0055] 图5是本发明一个实施例提供的流计算方法的方法流程图；
- [0056] 图6是本发明一个实施例提供的流计算方法的原理示意图；
- [0057] 图7是本发明另一个实施例提供的流计算方法的方法流程图；
- [0058] 图8A是本发明另一个实施例提供的流计算方法的方法流程图；
- [0059] 图8B是本发明另一个实施例提供的流计算方法的原理示意图；
- [0060] 图8C是本发明另一个实施例提供的流计算方法的方法流程图；
- [0061] 图8D是本发明另一个实施例提供的流计算方法的方法流程图；
- [0062] 图8E是本发明另一个实施例提供的流计算方法的方法流程图；
- [0063] 图9A是本发明一个实施例提供的流计算方法在具体实施时的原理示意图；
- [0064] 图9B是本发明另一实施例提供的流计算方法在具体实施时的原理示意图；
- [0065] 图10是本发明另一个实施例提供的流计算装置的结构方框图；
- [0066] 图11是本发明另一个实施例提供的流计算系统的结构方框图。

具体实施方式

[0067] 为使本发明的目的、技术方案和优点更加清楚，下面将结合附图对本发明实施方式作进一步地详细描述。图2A示出了本发明一个实施例提供的流计算系统的结构示意图。示意性的，该流计算系统是一个分布式计算系统，该分布式计算系统包括：终端220、管理节点240和多个工作节点260。

[0068] 终端220是诸如手机、平板电脑、膝上型便携计算机和台式计算机之类的电子设备，本实施例对终端220的硬件形式不加以限定。终端220中运行有客户端，客户端用于提供用户与分布式计算系统之间的人机交互入口。客户端具有根据用户的输入，获取输入通道描述信息、若干条SQL规则和输出通道描述信息的能力。

[0069] 可选地，客户端是分布式计算系统提供的原生客户端，或者，客户端是由用户自行开发的客户端。

[0070] 终端220通过有线网络、无线网络或专用硬件接口与管理节点240相连。

[0071] 管理节点240是一台服务器或几台服务器的组合，本实施例对管理节点240的硬件形式不加以限定。管理节点240是用于分布式计算系统中对各个工作节点260进行管理的节点。可选地，管理节点240用于对各个工作节点260进行资源管理、主备管理、应用管理和任务管理中的至少一种。资源管理是指对各个工作节点260中的计算资源进行管理；主备管理是指对各个工作节点260在发生故障时，实现主备切换管理；应用管理是指对运行在分布式计算系统上的至少一个流计算应用进行管理；任务管理是指对于一个流计算应用中的各个算子的计算任务进行管理。在不同的流计算系统中，管理节点240可能具有不同的名称，比如，主控节点(master node)。

[0072] 管理节点240通过有线网络、无线网络或专用硬件接口与工作节点260相连。

[0073] 工作节点260是一台服务器或几台服务器的组合，本实施例对工作节点260的硬件形式不加以限定。可选地，工作节点260中运行有流计算应用中的算子。每个工作节点260负责一个或多个算子的计算任务。比如，工作节点260中的每个进程用于负责一个算子的计算任务。

[0074] 当存在多个工作节点260时,多个工作节点260之间通过有线网络、无线网络或专用硬件接口相连。

[0075] 可以理解的是,在虚拟化场景下,流计算系统的管理节点240和工作节点260也可以由运行在通用硬件上的虚拟机来实现。图2B示出了本发明另一个实施例提供的流计算系统的结构示意图。示意性的,该流计算系统包括:若干台计算设备22所构成的分布式计算平台,每个计算设备22中运行有至少一个虚拟机,每个虚拟机是一个管理节点240或一个工作节点260。

[0076] 管理节点240和工作节点260是位于同一计算设备22上的不同虚拟机(如图2B中所示)。可选地,管理节点240和工作节点260是位于不同计算设备22上的不同虚拟机。

[0077] 可选地,每个计算设备22上运行不止一个工作节点260,每个工作节点260是一个虚拟机。每个计算设备22上所能够运行的工作节点260的数量,视计算设备22的计算能力所决定。

[0078] 可选地,各个计算设备22之间通过有线网络、无线网络或专用硬件接口相连。可选地,专用硬件接口是光纤、预定接口类型的电缆等。

[0079] 也即,本发明实施例不限定管理节点240是物理实体还是逻辑实体,也不限定工作节点260是物理实体还是逻辑实体。下面对管理节点240的结构和功能做进一步说明。

[0080] 图3A示出了本发明一个实施例提供的管理节点240的结构图。管理节点240包括:处理器241、网络接口242、总线243和存储器244。

[0081] 处理器241通过总线243分别与网络接口242、存储器244相连。

[0082] 网络接口242用于实现与终端220和工作节点260的通信。

[0083] 处理器241包括一个或一个以上处理核心。处理器241通过运行操作系统或应用程序模块,实现流计算系统中的管理功能。

[0084] 可选地,存储器244可存储操作系统245、至少一个功能所需的应用程序模块25。应用程序模块25包括获取模块251、生成模块252和执行模块253等。

[0085] 获取模块251,用于从客户端获取输入通道描述信息、SQL语句和输出通道描述信息。

[0086] 生成模块252,用于根据输入通道描述信息、SQL语句和输出通道描述信息生成流程图,流程图用于定义执行流计算任务的各个算子的计算逻辑以及各个算子之间数据流的输入输出关系。

[0087] 执行模块253,根据流程图控制工作节点执行流计算任务。

[0088] 此外,存储器244可以由任何类型的易失性或非易失性存储设备或者它们的组合实现,如静态随机存取存储器(SRAM),电可擦除可编程只读存储器(EEPROM),可擦除可编程只读存储器(EPROM),可编程只读存储器(PROM),只读存储器(ROM),磁存储器,快闪存储器,磁盘或光盘。

[0089] 本领域技术人员可以理解,图3A中所示出的结构并不构成管理节点240的限定,可以包括比图示更多或更少的部件或组合某些部件,或者不同的部件布置。

[0090] 图3B示出了一种虚拟化场景下的管理节点240的实施例,如图3B所示,管理节点240为运行在计算设备22上的虚拟机(Virtual Machine,VM)224。其中,计算设备22包括硬件层221,运行在硬件层221之上的虚拟机监视器(Virtual Machine Monitor,VMM)222,以及

运行在VMM 222之上的宿主机Host 223和若干虚拟机VM,其中,硬件层221包括但不限于:I/O设备、中央处理器(Central Processing Unit,CPU)和内存Memory.VM中运行有可执行程序,VM通过运行该可执行程序,并在程序运行的过程中通过宿主机Host 223来调用硬件层221的硬件资源,以实现上述获取模块251、生成模块252和执行模块253的功能。具体而言,获取模块251、生成模块252和执行模块253可以以软件模块或函数的形式被包含在上述可执行程序中,VM 224通过调用硬件层221中的CPU、Memory等资源,以运行该可执行程序,从而实现获取模块251、生成模块252和执行模块253的功能。

[0091] 结合图2A-2B和图3A-3B,下面对流计算系统执行流计算的整体过程进行介绍。图4示出了本发明一个实施例提供的流计算过程的原理示意图。整个流计算过程涉及数据生产系统41、流计算系统42和数据消费系统43。

[0092] 数据生产系统41用于产生数据。视不同的实施环境,数据生产系统41可以是金融系统、网络监控系统、生产制造系统、Web应用系统、传感检测系统等。

[0093] 可选地,数据生产系统41产生的数据的存储形式包括但不限于:文件、网络数据包、数据库中的至少一种。本发明实施例对数据的存储形式不加以限定。

[0094] 可选地,在硬件层面,数据生产系统41通过网络、光纤、硬件接口卡之类的硬件线路与流计算系统42相连。在软件层面,数据生产系统41通过输入通道411与流计算系统42相连。输入通道411是一种将来自数据生产系统41的数据流输入流计算系统42中的流图的逻辑通道,该逻辑通道用于实现数据生产系统41和流计算系统42之间在传输路径、传输协议、数据格式、数据编/解码方式等方面的对接。

[0095] 流计算系统42中通常包括由多个算子所构成的流图。该流图可认为是一个流计算应用。该流图中包括:源算子421、至少一个中间算子422和目的算子423。源算子421用于从数据生产系统41中接收输入数据流,源算子42还用于将输入数据流发送给中间算子422;中间算子422用于将输入数据流进行计算,将计算得到的输出数据流输入至下一级中间算子422或者目的算子423;目的算子423用于向数据消费系统43发送输出数据流。上述各个算子被图2所示的管理节点调度,以分布式的形式运行在图2所示的多个工作节点260中,每个工作节点260中运行有至少一个算子。

[0096] 可选地,在硬件层面,流计算系统42通过网络、光纤、硬件接口卡之类的硬件线路与数据消费系统43相连。在软件层面,流计算系统42通过输出通道421与数据消费系统43相连。输出通道421是一种将流计算系统42的输出数据流输出至数据消费系统43的逻辑通道,该逻辑通道用于实现流计算系统42和数据消费系统43之间在传输路径、传输协议、数据格式、数据编/解码方式等方面的对接。

[0097] 数据消费系统43用于对流计算系统42所计算出的输出数据流进行利用。数据消费系统43对输出数据流进行持久化存储或二次利用。比如,数据消费系统43是推荐系统,该推荐系统根据输出数据流向用户推荐感兴趣的网页、文本、音频、视频和购物信息等。

[0098] 其中,流计算系统42中的流图由用户通过客户端44进行生成、部署或调整。

[0099] 在本发明实施例中,流计算系统提供了一种利用SQL语句来构建流图的流图构建方式。示意性的,图5示出了本发明一个实施例提供的流计算方法的流程图。本实施例以该流计算方法应用于图2A-2B和图3A-3B所示的管理节点中来举例说明。该方法包括:

[0100] 步骤501,管理节点从客户端获取输入通道描述信息、SQL语句和输出通道描述信

息；

[0101] 用户通过客户端向管理节点发送输入通道描述信息、SQL语句和输出通道描述信息。

[0102] 输入通道描述信息用于定义输入通道,或者说,输入通道描述信息用于描述输入数据流的输入方式,或者说,输入通道描述信息用于描述输入通道的构建信息。输入通道是用于将来自数据生产系统的数据流输入流图的逻辑通道。

[0103] 可选地,输入通道描述信息包括:传输介质信息、传输路径信息、数据格式信息和数据解码方式信息中的至少一种。示意性的,一条输入通道描述信息包括:以太网介质、网络之间互连协议(Internet Protocol,IP)地址及端口号,传输控制协议(Transmission Control Protocol,TCP)数据包,默认解码方式;另一条输入通道描述信息包括:文件存储路径、Excel文件。

[0104] SQL语句用于定义流图中每个算子的计算逻辑,以及每个算子的输入数据流和输出数据流。可选地,每个算子存在至少一个输入数据流,每个算子存在至少一个输出数据流。

[0105] 输出通道描述信息用于定义输出数据流,或者说,输出通道描述信息用于描述输出数据流的输出方式,或者说,输出通道描述信息用于描述输出通道的构建信息。输出通道是用于将所述流图的输出数据流输出至数据消费系统的逻辑通道。

[0106] 可选地,输出通道描述信息包括:传输介质信息、传输路径信息、数据格式信息和数据编码方式信息中的至少一种。示意性的,一条输出通道描述信息包括:文件存储路径、CSV文件。

[0107] 管理节点接收客户端发送的输入通道描述信息、SQL和输出通道描述信息。

[0108] 步骤502,管理节点根据输入通道描述信息、SQL语句和输出通道描述信息生成流图,流图用于定义流计算中的各个算子的计算逻辑以及各个算子之间数据流的输入输出关系;

[0109] 可选地,SQL语句包括若干条SQL规则,每条SQL规则用于定义一个逻辑算子的计算逻辑,以及该算子的输入数据流和输出数据流。每条SQL规则包括至少一条SQL子语句。

[0110] 可选地,每个算子具有至少一个输入数据流,每个算子具有至少一个输出数据流。

[0111] 可选地,一个可执行的流图中,包括:源算子(Source)、中间算子和目标算子(Sink)。源算子用于接收来自数据生产系统的输入数据流,以及将输入数据流输入至中间算子中。中间算子用于对来自源算子的输入数据流进行计算,或者,中间算子用于对来自其他中间算子的输入数据流进行计算。目标算子用于根据来自中间算子的计算结果向数据消费系统发送输出数据流。

[0112] 步骤503,管理节点根据流图控制工作节点执行流计算任务。

[0113] 管理节点根据流图控制流计算系统中各个工作节点执行流计算任务。这里的“流图”应当被理解为一个可执行的流应用。

[0114] 可选地,管理节点将生成的流图调度至各个工作节点中进行分布式执行,多个工作节点根据流图对来自数据生产系统的输入数据流进行流计算,得到最终的输出数据流,并输出给数据消费系统。

[0115] 综上所述,本实现方式提供的流计算方法,通过由管理节点根据输入通道描述信

息、SQL语句和输出通道描述信息生成可执行的流图,然后由管理节点根据流图控制工作节点执行流计算;解决了目前的流计算系统通过IDE提供的基本算子来构建流图时,每个基本算子的功能被划分为非常细的粒度,导致生成的流图的整体计算性能较差的问题;达到了流计算系统支持SQL语句来构建流图,SQL是较为常见的数据库管理语言,用户使用SQL语句来构建流图仍然非常易用的效果,另一方面,由用户利用SQL语言的编程语言特性,采用SQL语句对流图的处理逻辑进行定义,由管理节点按照SQL语句所定义的处理逻辑动态生成具有合理数量的算子的流图,从而提高流图的整体计算性能。

[0116] 为了更清楚地理解图5实施例所提供的流计算方法的计算原理,请结合参考图6,从用户面来讲,需要用户配置输入通道描述信息61a、配置与业务有关的SQL规则62a、配置输出通道描述信息63a;从管理节点来讲,管理节点根据输入通道描述信息从数据生产系统引入输入数据流61b、通过SQL语句构建流图中的算子62b、根据输出通道描述信息向数据消费系统发送输出数据流63b;从工作节点来讲,需要执行管理节点生成的流计算应用中的源算子Source、中间算子CEP和目标算子Sink。

[0117] 上述步骤502可由若干个更为细分的步骤实现,在可选的实施例中,上述步骤502可被替代实现成为步骤502a和步骤502b,如图7所示:

[0118] 步骤502a,管理节点根据输入通道描述信息、若干条SQL规则和输出通道描述信息生成第一流图,第一流图包括若干个逻辑节点;

[0119] 步骤502b,管理节点将第一流图中的各个逻辑节点进行划分,以得到若干个逻辑节点组;在预设算子库中选择每个逻辑节点组对应的公共算子,并根据选择出的公共算子生成第二流图;第二流图中的每个算子用于实现该算子对应的逻辑节点组中的一个或多个逻辑节点的功能。

[0120] 可选地,第一流图是逻辑层面的临时流图,第二流图是代码层面的可执行流图。第一流图是根据SQL中的若干个SQL规则进行一层编译后所得到的临时流图;第二流图是根据第一流图进行二层编译后所得到的可执行流图。第二流图中的算子可被管理调度分配至工作节点中执行。

[0121] 管理节点在获取到输入通道描述信息、若干条SQL规则和输出通道描述信息后,先经过一层编译得到第一流图,第一流图包括通过有向边相连的源逻辑节点、若干个中间逻辑节点和目标逻辑节点。第一流图包括若干个逻辑层面的节点。

[0122] 然后,管理节点对第一流图中的各个逻辑节点进行划分,利用预设算子库中的公共算子,将第一流图中的各个逻辑节点组进行二层编译得到第二流图,第二流图中的每个算子用于实现第一流图中被划分为同一逻辑节点组的各个逻辑节点。

[0123] 公共算子是预先设置的用于实现某一种功能或某几种功能的通用型算子。

[0124] 示意性的,一个算子用于实现一个源逻辑节点的功能;或者,一个算子用于实现一个或者多个中间逻辑节点的功能;或者,一个算子用于实现一个目标逻辑节点的功能。

[0125] 示意性的,一个算子用于实现一个源逻辑节点和一个中间逻辑节点的功能;或者,一个算子用于实现一个源逻辑节点和多个中间逻辑节点的功能;或者,一个算子用于实现多个中间逻辑节点的功能;或者,一个算子用于实现一个中间逻辑节点和一个目的节点的功能;或者,一个算子用于实现多个中间逻辑节点和一个目的节点的功能。

[0126] 在对第一流图中的各个逻辑节点进行划分时,管理节点可按照负载均衡、算子并

发度、各个逻辑节点之间的亲密度和各个逻辑节点之间的互斥性中的至少一种因素进行逻辑节点的划分。

[0127] 当管理节点按照负载均衡进行划分时,管理节点参考每个算子的计算能力和每个逻辑节点所需要消耗的计算资源,将各个逻辑节点进行划分,使得每个算子所承担的计算量相对均衡。比如,一个算子的计算能力是100%,逻辑节点A所需要消耗的计算资源是30%,逻辑节点B所需要消耗的计算资源是40%,逻辑节点C所需要消耗的计算资源是50%,逻辑节点D所需要消耗的计算资源是70%,则将逻辑节点A和逻辑节点D划分至同一个逻辑节点组,将逻辑节点B和逻辑节点C划分至另一个逻辑节点组。

[0128] 当管理节点按照算子并发度进行划分时,管理节点获取每个输入数据流的数据流大小,根据每个输入数据流的数据流大小确定用于处理该输入数据流的逻辑节点的数量,使得每个输入数据流的计算速度保持相同或相似。

[0129] 当管理节点按照各个逻辑节点之间的亲密度来进行划分时,管理节点根据输入数据流的类型和/或逻辑节点之间的依赖关系计算逻辑节点之间的亲密度,然后将亲密度较高的逻辑节点划分为同一逻辑节点组。比如,输入数据流1同时是逻辑节点A和逻辑节点D的输入数据流,则逻辑节点A和逻辑节点D之间的亲密度较高,将逻辑节点A和逻辑节点D划分至同一逻辑节点组由同一个算子实现,能够减少各个算子之间的数据流传输量;又比如,逻辑节点A的输出数据流是逻辑节点B的输入数据流,逻辑节点B依赖于逻辑节点A,则逻辑节点A和逻辑节点B之间的亲密度较高,将逻辑节点A和逻辑节点B划分至同一逻辑节点组由同一个算子实现,也能够减少各个算子之间的数据流传输量。

[0130] 当管理节点按照各个逻辑节点之间的互斥性来进行划分时,管理节点检测各个逻辑节点之间的运算逻辑是否存在互斥性,当两个逻辑节点之间的运算逻辑存在互斥性时,将两个逻辑节点划分至不同的逻辑节点组。由于分布式计算系统的基础是多算子之间的并发与协作,这就不可避免地涉及到多个算子对共享资源的互斥访问,为了避免访问冲突,需要将存在互斥性的两个逻辑节点划分至不同的逻辑节点组中。

[0131] 综上所述,本实施例提供的流计算方法,用户只需要在逻辑层面编写SQL规则,由管理节点根据SQL规则生成第一流图,第一流图包括若干个逻辑节点,然后再由管理节点通过预设算子库将第一流图中各个逻辑节点进行划分后,得到若干个逻辑节点组,每个逻辑节点组被转换为第二流图中的一个算子,第二流图中的每个算子用于实现属于同一逻辑节点组中的各个逻辑节点,使得用户不需要具有流式编程思维,也不需要关心算子的划分逻辑,只需要在逻辑层面上编写SQL规则即可构建流图,由管理节点自行生成第二流图中的算子,从而减少了用户构建流计算应用时的代码编辑工作,降低用户构建流计算应用的复杂度。

[0132] 下文采用图8A实施例对上述流计算方法进行更为详细的示例和阐述。

[0133] 图8A示出了本发明另一个实施例提供的流计算方法的流程图。本实施例以该流计算方法应用于图2所示的流计算系统中来举例说明。该方法包括:

[0134] 步骤801,管理节点从客户端获取输入通道描述信息、SQL语句和输出通道描述信息;

[0135] 一、输入通道描述信息用于定义输入通道,输入通道是将来自数据生产系统的数据流输入流图的逻辑通道。

[0136] 以输入通道描述信息采用XML文件方式为例,一个示意性的输入通道描述信息如下:

```

    <channel name="tcp_channel_xdr" type="in"> //通道名称 tcp_channel_xdr,
    类型输入
        <transfers type= "tcp" > //传输类型: tcp
            <mode>server</mode> //传输模式: 服务器
            <addr>127.0.0.1: 8080; </ip> //传输地址: 127.0.0.1: 8080
        </transfers>
        <! --全局数据流格式定义, 及编解码格式定义-->
        <schemadep>
            <schema name= "XDR" type= "binary"> //模板名称 "XDR", 类型: 二
            进制文件
                <attribute name= "MSISON" type= "string" length= "12" /> //属性
            名称: MSISON 类型: string 长度: 12
                < attribute name= "HOST" type= "string" length= "4" /> //属性名
            称: HOST 类型: string 长度: 4
                < attribute name= "Case ID" type= "unit32" /> //属性名称: Case ID
            类型: 32 位整型
        </schema>
    </schemadep>
    <! --输入数据流定义-->
    <in>
        <stream name= "cau_xdr" decode= "default" schema= "XDR" /> //输入
        数据流名称: cau_xdr 解码方式: 默认, 模板名称: XDR
    </in>
    </channel>;

```

[0139] 本发明实施例对输入通道描述信息的具体形式不加以限定,上述例子仅为示意性说明。

[0140] 可选地,来自数据生产系统的输入数据流是TCP或UDP数据流,文件,数据库,分布式文件系统(Hadoop Distributed File System,HDFS)等。

[0141] 二、SQL用于定义流图中每个算子的计算逻辑,以及每个算子的输入数据流和输出数据流。

[0142] SQL包括:数据定义语言(Data Definition Language,DLL)和数据操纵语言(Data Manipulation Language,DML)。通过SQL来定义流图中的各个算子时,通常采用DLL语言定义输入数据流和/或输出数据流,比如创建(create)子语句;采用DML语言定义计算逻辑,比如,插入(insert into)子语句、选择(Select)子语句。

[0143] 为了定义流图中的多个算子,SQL通常包括多条SQL规则,每条SQL规则包括至少一条SQL子语句,每条SQL规则用于定义流图中的一个逻辑节点。

[0144] 示意性的,一组典型的SQL规则包括:

[0145] Insert into B...

[0146] Select...

[0147] From A...

[0148] Where...

[0149] 在数据库领域中,Insert into子语句是SQL中向数据表中插入数据的语句,Select子语句是SQL中用于从数据表中选取数据的语句,from子语句是SQL中用于从数据表中读取数据的语句,Where子语句是在需要按照条件从数据表中选取数据时,添加在Select子语句中的条件语句。在上述例子中,输入数据流为A,输出数据流为B。

[0150] 在本实施例的SQL中,Insert into子语句被转用于用于定义输出数据流的语句,Select子语句被转用于表示计算逻辑的语句,from子语句被转用于用于定义输入数据流的语句,Where子语句被转用于选择数据的语句。

[0151] 示意性的,用户输入用于配置一个流图的若干条SQL规则包括如下:

```
Create stream s_edr(TriggerType uint32, MSISDN string, QuotaName string,
QuotaConsumption uint32, QuotaBalance uint32, CaseID uint32)
as select * from tcp_channel_edr.edr_event;           //SQL 规则 1
```

```
Create stream s_xdr(MSISDN string, Host string, CaseID uint32, CI uint32,
App_Category uint32, App_sub_Category uint32, Up_Throughput uint32,
Down_Throughput uint32)
as select * from tcp_channel_xdr.xdr_event;           //SQL 规则 2
```

[0152]

```
insert into temp1
select *from s_edr as a
where a.QuotaName= ' GPRS ' and a.QuotaConsumption * 10 >
=a.QuotaBalance * 8;                                //SQL 规则 3
```

```
insert into file_channel_result1.cep_result
select b.*,1 as Fixnum
from s_xdr as a,temp1.win:time_sliding(15 sec) as b
```



```
where a.MSISON= b.MSISDN; //SQL 规则 4
```

```
insert into file_channel_result2.cep_result
```

```
[0153] select MSISDN, App_Category, App_sub)_category;
        sum ( Up_Throughput+Down_Throughput ) as Throughput
from s_xdr.win:time_tumbling(5 min)
group by MSISDN, App_Category, APP_Sub_Category //SQL 规则 5
```

[0154] 其中,SQL规则1的输入数据流是tcp_channel_edr,输出数据流是s_edr;SQL规则2的输入数据流是tcp_channel_xdr,输出数据流是s_xdr;SQL规则3的输入数据流是tcp_channel_edr,输出数据流是s;SQL规则4的输入数据流是s_xdr和temp1,输出数据流是file_channel_result1;SQL规则5的输入数据流是s_xdr,输出数据流是file_channel_result2。

[0155] 三、输出通道描述信息用于定义输出通道,输出通道是向数据消费系统发送输出数据流的逻辑通道。

[0156] 以输出通道描述信息采用XML文件方式为例,一个示意性的输入通道描述信息如下:

```
<channel name= "file_channel_result" type= "out" > //通道名称
file_channel_result, 类型:输出
    <parameter> //参数
        <type>file</type> //类型: 文件
        <mode>csv</mode> //传输模式: csv 文件
        <line_terminator>\n</line_terminator> //行终止符: \n
        <file_name>/home/demo/result.csv</file_name> // 文 件 名 :
[0157] /home/demo/result.csv
    </ parameter >

    <schema name= "RESULT_OUT" type= "text" delimiter " ," > //模板名
称: RESULT_OUT, 类型: text , 分界符: ,
        <attribute name= "TriggerType" type= "uint32" /> //属性名称:
TriggerType 类型: uint32
```

```

        < attribute name= "MSISDN" type= "string" /> //属性名称: MSISDN
    类型: string
        < attribute name= "Case ID" type= "unit32" /> //属性名称: Case ID
    类型: 32 位整型
    </schema>
[0158] <! --输出数据流定义-->
    <out>
        <stream name= "outevent" schema= "RESULT_OUT" /> //输出流名称:
    outevent , 模板名称: RESULT_OUT
    </out>
    </channel>。

```

[0159] 可选地,来自数据生产系统的输入数据流是TCP或UDP数据流,文件,数据库,分布式文件系统(Hadoop Distributed File System,HDFS)等。

[0160] 第一流图是包括源逻辑节点、中间逻辑节点和目标逻辑节点的临时性流图。第一流图是逻辑层面的流图。该第一流图的生成过程,可包括如下步骤802至步骤805:

[0161] 步骤802,管理节点根据输入通道描述信息生成源逻辑节点;

[0162] 可选地,源逻辑节点用于接收来自数据生产系统的输入数据流。通常,每个源逻辑节点用于接收来自数据生产系统的一个输入数据流。

[0163] 步骤803,管理节点根据SQL语句中的每条SQL规则,根据SQL规则中的选择子语句生成中间逻辑节点;

[0164] 可选地,对于每个SQL规则,根据该SQL规则中的select子语句所限定的计算逻辑,生成中间逻辑节点。

[0165] 比如,根据SQL规则1中的select语句,生成用于对输入数据流tcp_channel_edr进行计算的中间逻辑节点。又比如,根据SQL规则2中的select语句,生成用于对输入数据流tcp_channel_xdr进行计算的中间逻辑节点。

[0166] 步骤804,管理节点根据输出通道描述信息生成目标逻辑节点;

[0167] 可选地,目标逻辑节点用于向数据消费系统发送输出数据流。通常,每个目标逻辑节点用于输出一个输出数据流。

[0168] 步骤805,管理节点根据SQL规则中的输入子语句和输出子语句,生成源逻辑节点与中间逻辑节点、中间逻辑节点与中间逻辑节点、中间逻辑节点与目标逻辑节点之间的有向边。

[0169] 根据SQL规则中的from子语句,生成与该SQL规则对应的中间逻辑节点的输入边。该输入边的另一端与源逻辑节点相连,或者,该输入边的另一端与其它中间逻辑节点相连。

[0170] 根据SQL规则中的Insert into子语句,生成与该SQL规则对应的中间逻辑节点的输出边。该输出边的另一端与其它中间逻辑节点相连,或者,该输出边的另一端与目标逻辑节点相连。

[0171] 对于一个中间逻辑节点来讲,输入边是指向该中间逻辑节点的有向边,输出边是从该中间逻辑节点指向其它中间逻辑节点或目标逻辑节点的有向边。

[0172] 示意性的,如图8B所示,第一流图包括:第一源逻辑节点81、第二源逻辑节点82、第一中间逻辑节点83、第二中间逻辑节点84、第三中间逻辑节点85、第四中间逻辑节点86、第五中间逻辑节点87、第一目标逻辑节点88和第二目标逻辑节点89。

[0173] 第一源逻辑节点81的输出数据流tcp_channel_edr,是第一中间逻辑节点83的输入数据流。

[0174] 第二源逻辑节点82的输出数据流tcp_channel_xdr,是第二中间逻辑节点84的输入数据流。

[0175] 第一中间逻辑节点83的输出数据流s_edr,是第三中间逻辑节点85的输入数据流。

[0176] 第三中间逻辑节点85的输出数据流temp1,是第四中间逻辑节点86的输入数据流。

[0177] 第二中间逻辑节点84的输出数据流s_xdr,是第四中间逻辑节点86的输入数据流。

[0178] 第二中间逻辑节点84的输出数据流s_xdr,是第五中间逻辑节点87的输入数据流。

[0179] 第四中间逻辑节点86的输出数据流file_channel_result1,是第一目标逻辑节点88的输入数据流。

[0180] 第五中间逻辑节点87的输出数据流file_channel_result2,是第二目标逻辑节点89的输入数据流。

[0181] 需要说明的是,本实施例对步骤802、步骤803和步骤804互相之间的执行先后顺序不加以限定,可选地,上述步骤802、步骤803和步骤804是并行执行的步骤,或者,上述步骤802、步骤803和步骤804是先后串行执行的步骤。

[0182] 第二流图是一个可执行的流计算应用,第二流图是代码层面的流图。该第二流图的生成过程,可包括如下步骤806至步骤808:

[0183] 步骤806,管理节点编译公共源算子以得到第二流图中的源算子;

[0184] 可选地,管理节点根据源逻辑节点在预设算子库中选择出公共源算子,根据公共源算子编译得到第二流图中的源算子;

[0185] 可选地,预设算子库中设置有一种或多种公共源算子,比如,对应于TCP协议的公共源算子、对应于用户数据报协议(User Datagram Protocol,UDP)协议的公共源算子、对应于文件类型A的公共源算子、对应于文件类型B的公共源算子、对应于数据库类型A的公共源算子和对应于数据库类型B的公共源算子等。

[0186] 可选地,管理节点将每个源逻辑节点划分为一个逻辑节点组,每个源逻辑节点实现成为一个源算子。

[0187] 管理节点根据第一流图中的源逻辑节点,从预设算子库中选择出对应的公共源算子进行编译,能够得到第二流图中的源算子。源算子用于接收来自数据生产系统的输入数据流。

[0188] 步骤807,管理节点在预设算子库中为每个包括中间逻辑节点的逻辑节点组选择出至少一个公共中间算子,编译选择出的公共中间算子以得到第二流图中的中间算子;

[0189] 可选地,管理节点将至少一个中间逻辑节点进行划分,得到若干个逻辑节点组;根据划分为同一逻辑节点组中的各个中间逻辑节点,在预设算子库中选择出与该逻辑节点组对应的公共中间算子,根据公共中间算子编译得到第二流图中的中间算子;

[0190] 可选地,预设算子库中设置有一种或多种公共计算算子,比如,用于实现乘法运算的公共中间算子、用于实现减法运算的公共中间算子、用于实现排序运算的公共中间算子、

用于筛选运算的公共中间算子等等。当然,同一个公共中间算子的功能可以为多种,也即,公共中间算子是具有多种计算功能的算子。当同一个公共中间算子的功能为多种时,能够在一个公共中间算子上实现多个逻辑节点。

[0191] 由于第一流图中每个中间逻辑节点的计算类型和/或计算量不同,管理节点根据负载均衡、并发度要求、各个逻辑节点之间的亲密度和各个逻辑节点之间的互斥性中的至少一个因素将每个中间逻辑节点进行划分,被划分至同一逻辑节点组各个中间逻辑节点通过预设算子库中的同一个公共中间算子进行编译,得到第二流图中的一个中间算子。

[0192] 比如,管理节点将两个计算量较少的中间逻辑节点划分为同一组;又比如,管理节点将中间逻辑节点A、中间逻辑节点B、中间逻辑节点C划分为同一组,其中,中间逻辑节点A的输出数据流是中间逻辑节点B的输入数据流,中间逻辑节点B的输出数据流是中间逻辑节点C的输入数据流;再比如,管理节点将具有相同输入数据流的中间逻辑节点A和中间逻辑节点D划分为同一组。

[0193] 步骤808,管理节点编译公共目标算子以得到第二流图中的目标算子;

[0194] 可选地,管理节点根据目标逻辑节点在预设算子库中选择出公共目标算子,根据公共目标算子编译得到第二流图中的目标算子。

[0195] 可选地,预设算子库中设置有一种或多种公共目的算子,比如,对应于TCP协议的公共目的算子、对应于UDP协议的公共目的算子、对应于文件类型A的公共目的算子、对应于文件类型B的公共目的算子、对应于数据库类型A的公共目的算子和对应于数据库类型B的公共目的算子等。

[0196] 可选地,管理节点将每个目标逻辑节点划分为一个逻辑节点组,每个目标逻辑节点实现成为一个目标算子。

[0197] 管理节点根据第一流图中的目标逻辑节点,从预设算子库中选择出对应的公共目标算子进行编译,能够得到第二流图中的目标算子。目标算子用于向数据消费系统发送最终的输出数据流。

[0198] 示意性的,参考图8B所示,将第一流图中的第一源逻辑节点81通过公共源算子进行编译,得到第一源算子source1;将第一流图中的第二源逻辑节点82通过公共源算子进行编译,得到第二源算子source2;将第一流图中的第一中间逻辑节点83至第五中间逻辑节点87划分为同一组,通过同一个公共中间算子进行编译,得到中间算子CEP;将第一流图中的第一目标逻辑节点通过公共目的算子进行编译,得到第一目的算子sink1;将第一流图中的第二目标逻辑节点通过公共目的算子进行编译,得到第二目的算子sink2。

[0199] 最终,第二流图包括:第一源算子source1、第二源算子source2、中间算子CEP、第一目的算子sink1和第二目的算子sink2。

[0200] 步骤809,管理节点根据源逻辑节点与中间逻辑节点、中间逻辑节点与中间逻辑节点、中间逻辑节点与目标逻辑节点之间的有向边,生成第二流图中的各个算子之间的有向边。

[0201] 管理节点根据第一流图中的各个有向边,对应地生成第二流图中的各个算子之间的有向边。

[0202] 至此,一个可执行的流图被生成。该流图也可被认为是一个流计算应用。

[0203] 需要说明的是,本实施例对步骤806、步骤807和步骤808互相之间的执行先后顺序

不加以限定,可选地,上述步骤806、步骤807和步骤808是并行执行的步骤,或者,上述步骤806、步骤807和步骤808是先后串行执行的步骤。

[0204] 步骤810,管理节点根据将第二流图中的各个算子调度至分布式计算系统中的至少一个工作节点中,该工作节点用于执行算子;

[0205] 分布式计算系统中包括有多个工作节点,管理节点按照自身决策的物理执行计划,将第二流图中的各个算子调度至多个工作节点中进行执行。每个工作节点用于执行至少一个算子。通常情况下,每个工作节点中运行有至少一个进程,每个进程用于执行一个算子。

[0206] 比如,第一源算子source1被调度至工作节点1、第二源算子source2被调度至工作节点2、中间算子CEP被调度至工作节点3、第一目的算子sink1和第二目的算子sink2被调度至工作节点4。

[0207] 为了解耦各个算子之间的数据流引用关系,本实施例还引入了订阅机制。

[0208] 步骤811,管理节点根据每个算子的输出数据流,生成与该算子对应的订阅发布信息,向该算子配置该订阅发布信息;

[0209] 订阅发布信息用于指示与当前算子对应的输出数据流的发布方式。

[0210] 管理节点根据当前算子的输出数据流、第二流图中的有向边和各个工作节点之间的拓扑结构,生成与当前算子对应的订阅发布信息。

[0211] 比如,第一源算子source1的输出数据流是tcp_channel_edr,在第二流图中与tcp_channel_edr对应的有向边指向中间算子CEP,工作节点1的网络接口3与工作节点3的网络接口4相连,则管理节点生成将输出数据流tcp_channel_edr从工作节点1的网络接口3以预定形式进行发布的订阅发布信息。然后,管理节点将该订阅发布信息下发给位于工作节点1中的第一源算子source1,第一源算子source1根据该订阅发布信息向外发布输出数据流tcp_channel_edr,此时,第一源算子source1并不需要关心下游的算子具体是哪一个算子,也不需要关心下游的算子位于哪个工作节点,只需要按照订阅发布信息从工作节点1的网络接口3发布输出数据流即可。

[0212] 步骤812,管理节点根据每个算子的输入数据流,生成与该算子对应的输入流定义信息,向该算子配置输入流定义信息;

[0213] 输入流定义信息用于指示与当前算子对应的输入数据流的接收方式。

[0214] 管理节点根据当前算子的输入数据流、第二流图中的有向边和各个工作节点之间的拓扑结构,生成与当前算子对应的订阅信息。

[0215] 比如,中间算子CEP的输入数据流包括tcp_channel_edr,在第二流图中与tcp_channel_edr对应的有向边来源于第一源算子Source1,工作节点1的网络接口3与工作节点3的网络接口4相连,则管理节点生成从网络接口4以预定形式进行接收的输入流定义信息。然后,管理节点将该输入流定义信息下发给位于工作节点3中的中间算子CEP,中间算子CEP根据该输入流定义信息接收输入数据流tcp_channel_edr,此时,中间算子CEP并不需要关心上游的算子具体是哪一个算子,上游的算子具体位于哪个工作节点,只需要按照输入流定义信息从工作节点3的网络接口4接收输入数据流即可。

[0216] 步骤813,工作节点对第二流图中的各个算子进行执行;

[0217] 各个工作节点根据管理节点的调度,对第二流图中的各个算子进行执行。比如,每

个进程用于负责一个算子的计算任务。

[0218] 综上所述,本实施例提供的流计算方法,通过由管理节点根据输入通道描述信息、SQL语句和输出通道描述信息生成可执行的流图,然后由管理节点根据流图控制工作节点执行流计算;解决了目前的流计算系统通过IDE提供的基本算子来构建流图时,每个基本算子的功能被划分为非常细的粒度,导致生成的流图的整体计算性能较差的问题;达到了流计算系统支持SQL语句来构建流图,SQL是较为常见的数据库管理语言,用户使用SQL语句来构建流图仍然非常易用的效果,另一方面,由用户利用SQL语言的编程语言特性,采用SQL语句对流图的处理逻辑进行定义,由管理节点按照SQL语句所定义的处理逻辑动态生成具有合理数量的算子的流图,从而提高流图的整体计算性能。

[0219] 还通过由管理节点将第一流图中的多个逻辑节点进行划分,将划分为同一组的各个逻辑节点通过同一个公共中间算子进行实现,不需要用户考虑负载均衡、并发执行、亲密度和互斥性等因素,由管理节点自行决策负载均衡、并发执行、亲密度和互斥性等因素来进行第二流图的生成,进一步地降低了用户生成第二流图时的难度,只需要用户具有通过SQL构建逻辑层面的第一流图的能力即可。

[0220] 还通过设置订阅机制,将第二流图中的各个算子的输入数据流和输出数据流之间的引用关系解耦,提供了在第二流图被执行后,用户仍然可以在流计算系统中动态调整第二流图中的各个算子的能力,提高了流计算应用的整体易用性和可维护性。

[0221] 当第二流图在流计算系统中被执行以后,随着业务功能在实际使用场景中的变更和调整,已经执行的第二流图也需要进行改变才能适应新的需求。与现有技术中通常需要重新构建第二流图不同的是,本发明实施例提供了对已执行的第二流图进行动态修改的能力,具体参考如下图8C至图8E。

[0222] 在第二流图被执行以后,用户还可对第二流图中的中间算子进行修改。如图8C所示:

[0223] 步骤814,客户端向管理节点发送第一修改信息;

[0224] 第一修改信息是对SQL规则进行修改的信息;或者说,第一修改信息携带有修改后的SQL规则。

[0225] 若第二流图中的中间算子需要修改,则客户端向管理节点发送用于对SQL规则进行修改的第一修改信息。

[0226] 步骤815,管理节点接收来自客户端的第一修改信息;

[0227] 步骤816,管理节点根据第一修改信息对第二流图中的中间算子进行增加、修改或删除。

[0228] 可选地,对一个原有的中间算子,替换为一个新的中间算子的修改过程,可以是将原有的中间算子进行删除,再增加新的中间算子。

[0229] 步骤817,管理节点向修改后的中间算子重新配置订阅发布信息 and/或输入流定义信息;

[0230] 可选地,若修改后的中间算子的输入数据流是新增的数据流或发生改变的数据流,则管理节点还需要重新向该中间算子配置输入流定义信息。

[0231] 若修改后的中间算子的输出数据流是新增的数据流或发生改变的数据流,则管理节点还需要重新向该中间算子配置订阅发布信息。

[0232] 综上所述,本实施例提供的流计算方法,通过客户端向管理节点发送第一修改信息,由管理节点根据第一修改信息对第二流图中的中间算子进行增加、修改或删除,为管理节点提供了动态调整第二流图中的中间算子的能力。

[0233] 在第二流图被执行以后,用户还可对第二流图中的源算子进行修改。如图8D所示:

[0234] 步骤818,客户端向管理节点发送第二修改信息;

[0235] 第二修改信息是对输入通道描述信息进行修改的信息;或者说,第二修改信息携带有修改后的输入通道描述信息。

[0236] 若第二流图中的源算子需要修改,则客户端向管理节点发送用于对输入通道描述信息进行修改的第二修改信息。

[0237] 步骤819,管理节点接收来自客户端的第二修改信息;

[0238] 步骤820,管理节点根据第二修改信息对第二流图中的源算子进行增加、修改或删除。

[0239] 可选地,对一个原有的源算子,替换为一个新的源算子的修改过程,可以是将原有的源算子进行删除,再增加新的源算子。

[0240] 步骤821,管理节点向修改后的源算子重新配置订阅发布信息。

[0241] 可选地,若修改后的源算子的输出数据流是新增的数据流或发生改变的数据流,则管理节点还需要重新向该源算子配置订阅发布信息。

[0242] 综上所述,本实施例提供的流计算方法,通过客户端向管理节点发送第二修改信息,由管理节点根据第二修改信息对第二流图中的源算子进行增加、修改或删除,为管理节点提供了动态调整第二流图中的源算子的能力。

[0243] 在第二流图被执行以后,用户还可对第二流图中的目标算子进行修改。如图8E所示:

[0244] 步骤822,客户端向管理节点发送第二修改信息;

[0245] 第二修改信息是对输入通道描述信息进行修改的信息;或者说,第二修改信息携带有修改后的输入通道描述信息。

[0246] 若第二流图中的源算子需要修改,则客户端向管理节点发送用于对输入通道描述信息进行修改的第二修改信息。

[0247] 步骤823,管理节点接收来自客户端的第二修改信息;

[0248] 步骤824,管理节点根据第二修改信息对第二流图中的源算子进行增加、修改或删除。

[0249] 可选地,对一个原有的源算子,替换为一个新的目标算子的修改过程,可以是将原有的目标算子进行删除,再增加新的目标算子。

[0250] 步骤825,管理节点向修改后的目标算子重新配置输入流定义信息。

[0251] 可选地,若修改后的目标算子的输入数据流是新增的数据流或发生改变的数据流,则管理节点还需要重新向该目标算子配置输入流定义信息。

[0252] 综上所述,本实施例提供的流计算方法,通过客户端向管理节点发送第三修改信息,由管理节点根据第三修改信息对第二流图中的目标算子进行增加、修改或删除,为管理节点提供了动态调整第二流图中的目标算子的能力。

[0253] 在一个具体的实施例中,如图9A所示,流计算系统向用户提供两种客户端:由流计

算系统提供的原生客户端92,和,由用户二次开发的客户端94。原生客户端92和二次开发的客户端94都提供有SQL应用程序编程接口(Application Programming Interface,API),该SQL API用于实现使用SQL语言来定义流图的功能。用户在原生客户端92或二次开发的客户端94录入输入/输出通道描述信息和SQL语句,原生客户端92或二次开发的客户端94将输入/输出通道描述信息和SQL语句发送给管理节点Master,也即图中的步骤1。

[0254] 管理节点Master通过App连接服务与原生客户端92或二次开发的客户端94建立连接。管理节点Master获取输入/输出通道描述信息和SQL语句,由SQL引擎96根据输入/输出通道描述信息和SQL语句生成可执行的流图,也即图中的步骤2。

[0255] 管理节点Master还包括流平台执行框架管理模块98,该流平台执行框架管理模块98用于实现资源管理、应用管理、主备管理和任务管理等管理事务。对于SQL引擎96所生成的一个可执行流图。由流平台执行框架管理模块98规划和决策该流图在各个工作节点Worker上的执行计划,也即图中的步骤3。

[0256] 各个工作节点Worker上的处理单元集(processing element container,PEC) 包括多个处理单元PE,每个PE用于调用可执行流图中的一个源算子Soucer、或者一个中间算子CEP、或者一个目标算子Slink。通过各个PE之间的协作,对可执行流图中的各个算子进行处理。

[0257] 图9B示出了本发明一个实施例提供的SQL引擎96在具体实施时的原理示意图。SQL引擎96在获取到输入/输出通道描述信息和SQL语句后,执行如下几个过程:

[0258] 1、SQL引擎96对SQL语句中的各个SQL规则进行解析;2、SQL引擎96根据解析结果生成临时的第一流图;3、SQL引擎96对第一流图中的各个逻辑节点按照负载均衡、亲密度和互斥性等因素进行划分,得到至少一个逻辑节点组,每个逻辑节点组包括一个或多个逻辑节点;4、SQL引擎96进行算子并发度计算的模拟,按照算子并发度计算的模拟结果对各个逻辑节点组进行调整;5、SQL引擎96根据调整后的各个逻辑节点组生成第二流图,属于同一个逻辑节点组中的各个逻辑节点被分配至第二流图中的一个可执行算子;6、SQL引擎96对第二流图中的各个可执行算子进行解析,分析每个算子的运算要求等信息;7、SQL引擎96对第二流图中的各个可执行算子生成逻辑执行计划;8、SQL引擎96对第二流图的逻辑执行计划进行代码编辑优化,生成物理执行计划。9、SQL引擎96向流平台执行框架管理模块98发送物理执行计划,由流平台执行框架管理模块98按照物理执行计划进行流计算应用的执行。

[0259] 其中,步骤1至步骤5属于一层编译过程,步骤6至步骤9属于二层编译过程。

[0260] 以下为本发明的装置实施例,该装置实施例与上述方法实施例对应,装置实施例中未详细阐述的细节,可参考上述方法实施例中的描述。

[0261] 图10示出了本发明一个实施例提供的流计算装置的结构方框图。该流计算装置可以通过专用硬件电路,或者,软硬件的组合实现成为管理节点240的全部或一部分。该流计算装置包括:获取单元1020、生成单元1040和执行单元1060。

[0262] 获取单元1020,用于实现上述步骤501、步骤801的功能;

[0263] 生成单元1040,用于实现上述步骤502、步骤502a、步骤502b、步骤802至步骤808的功能;

[0264] 执行单元1060,用于实现上述步骤503、步骤810至步骤812的功能。

[0265] 可选地,该装置还包括:修改单元1080;

[0266] 修改单元1080,用于实现上述步骤815至步骤825的功能。

[0267] 相关细节可结合参考图5、图6、图7、图8A、图8B、图8C、图8D和图8E所述的方法实施例。

[0268] 可选地,上述获取单元1020通过管理节点240的网络接口242以及处理器241执行存储器244中的获取模块251来实现。该网络接口242是以太网网卡、光纤收发器、通用串行总线(Universal Serial Bus,USB)接口或者其它I/O接口。

[0269] 可选地,上述生成单元1040通过管理节点240的处理器241执行存储器244中的生成模块252来实现。该生成单元1040所生成的流图是由多个算子所形成的可执行的分布式流计算应用,该分布式流计算应用中的每个算子可分派至不同的工作节点去执行。

[0270] 可选地,上述执行单元1060通过管理节点240的网络接口242以及处理器241执行存储器244中的执行模块253来实现。该网络接口242是以太网网卡、光纤收发器、USB接口或者其它I/O接口。换句话说,处理器241将流图中的各个算子通过网络接口242分派至不同的工作节点,然后由各个工作节点执行该算子的数据计算。

[0271] 可选地,上述修改单元1080通过管理节点240的处理器241执行存储器244中的修改模块(图中未示出)来实现。

[0272] 需要说明的是:上述实施例提供的流计算装置在生成流图并进行流计算时,仅以上述各功能模块的划分进行举例说明,实际应用中,可以根据需要而将上述功能分配由不同的功能模块完成,即将设备的内部结构划分成不同的功能模块,以完成以上描述的全部或者部分功能。另外,上述实施例提供的流计算装置与流计算方法的方法实施例属于同一构思,其具体实现过程详见方法实施例,这里不再赘述。

[0273] 图11示出了本发明一个实施例提供的流计算系统的结构方框图。该流计算系统包括:终端1120、管理节点1140和工作节点1160。

[0274] 终端1120,用于执行上述方法实施例中由终端或客户端所执行的步骤。

[0275] 管理节点1140,用于执行上述方法实施例中由管理节点所执行的步骤。

[0276] 工作节点1160,用于执行上述方法实施例中由工作节点所执行的步骤。

[0277] 上述本发明实施例序号仅仅为了描述,不代表实施例的优劣。

[0278] 本领域普通技术人员可以理解实现上述实施例的全部或部分步骤可以通过硬件来完成,也可以通过程序来指令相关的硬件完成,所述的程序可以存储于一种计算机可读存储介质中,上述提到的存储介质可以是只读存储器,磁盘或光盘等。

[0279] 以上所述仅为本发明的较佳实施例,并不用以限制本发明,凡在本发明的精神和原则之内,所作的任何修改、等同替换、改进等,均应包含在本发明的保护范围之内。

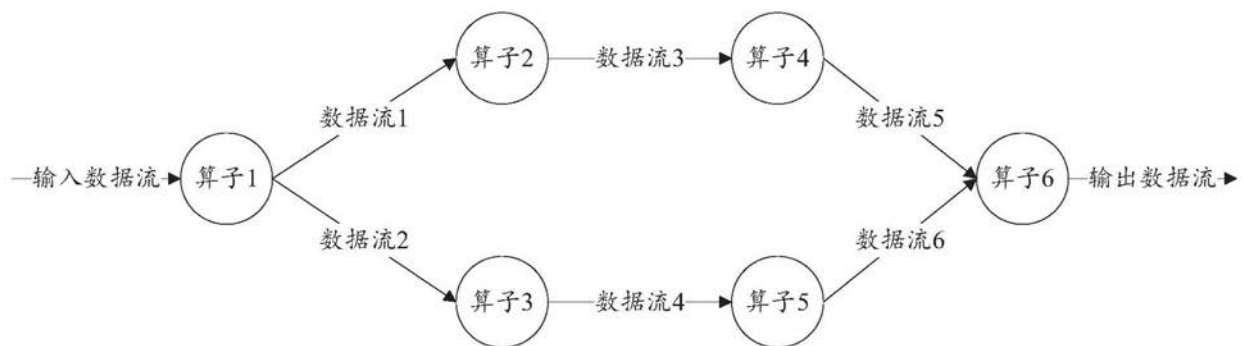


图1

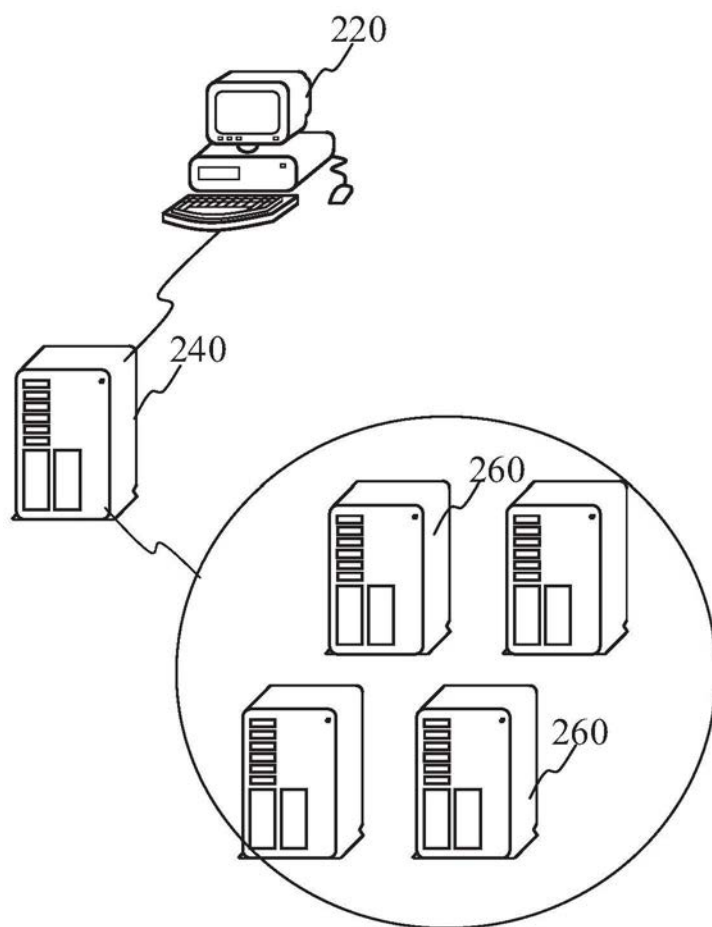


图2A

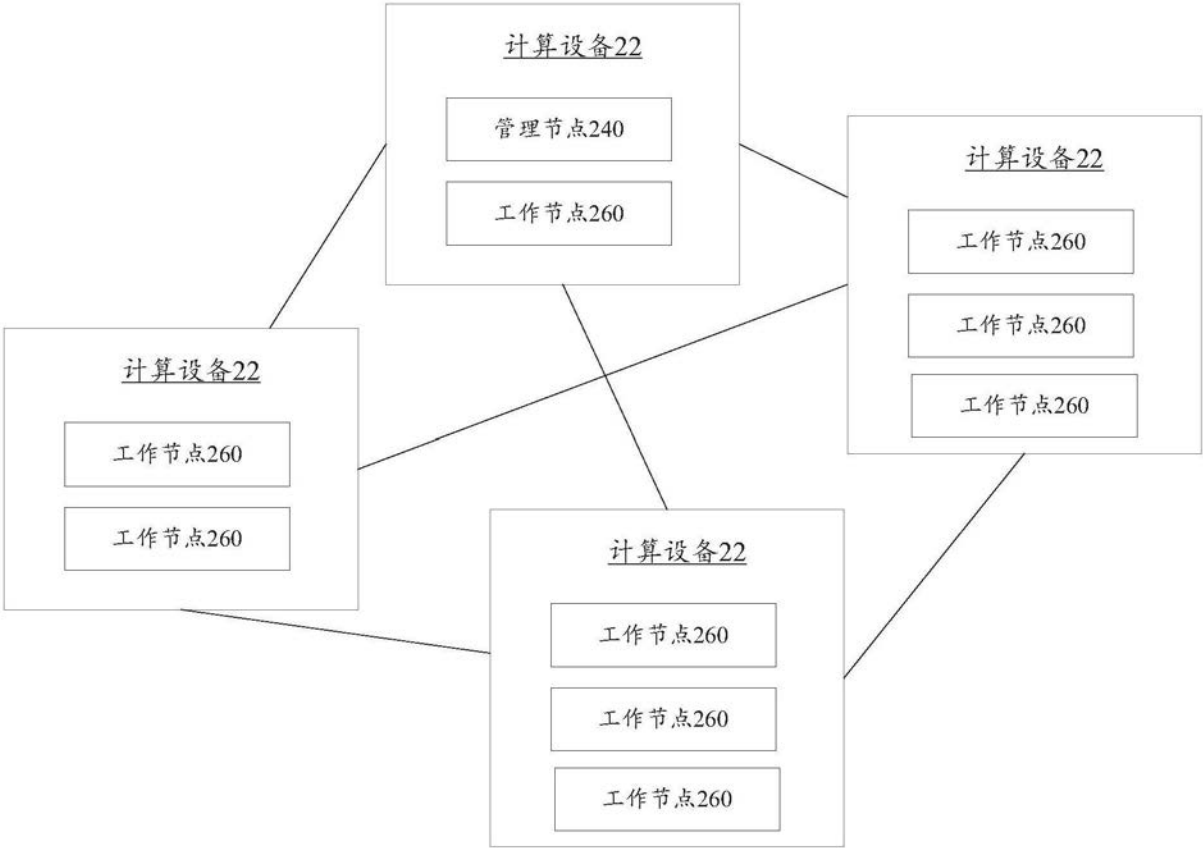


图2B

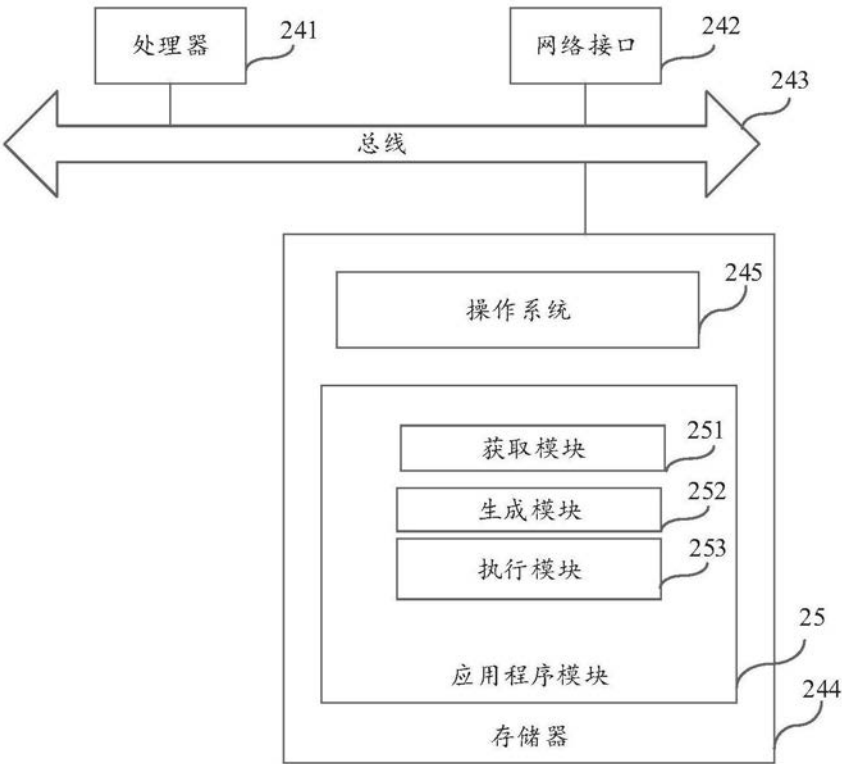


图3A

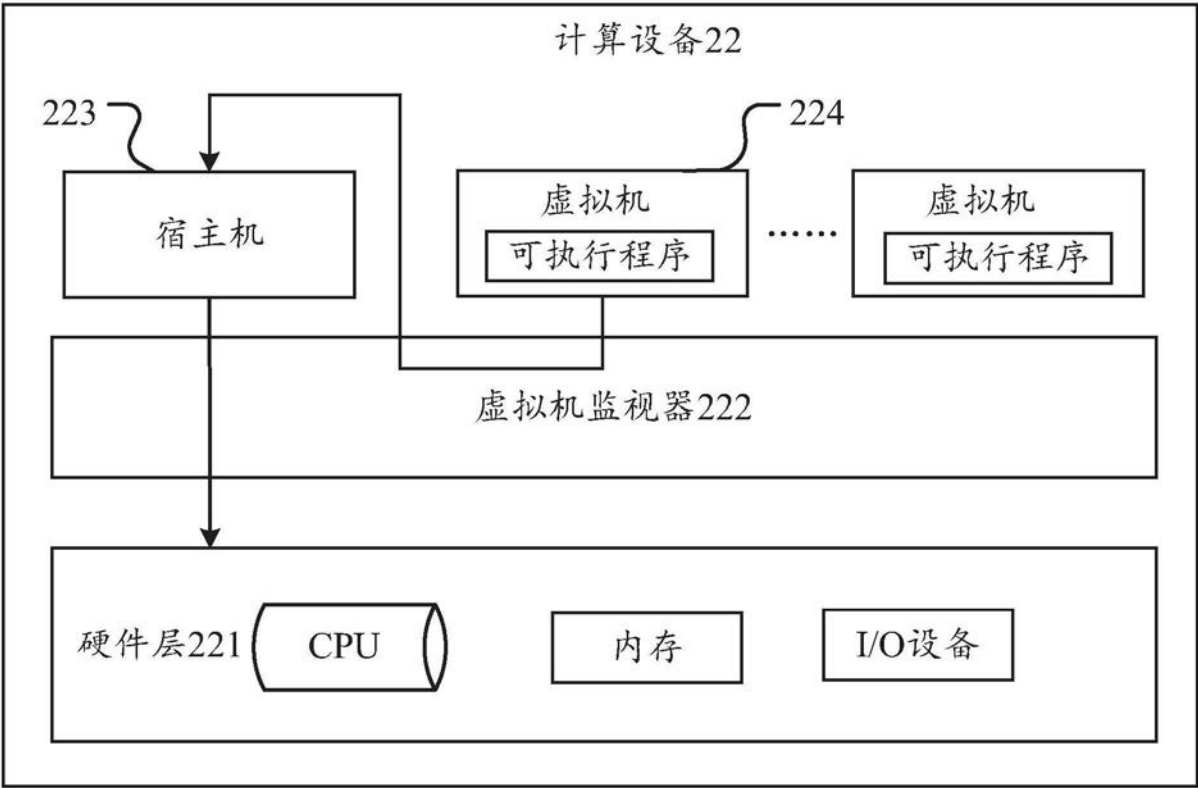


图3B

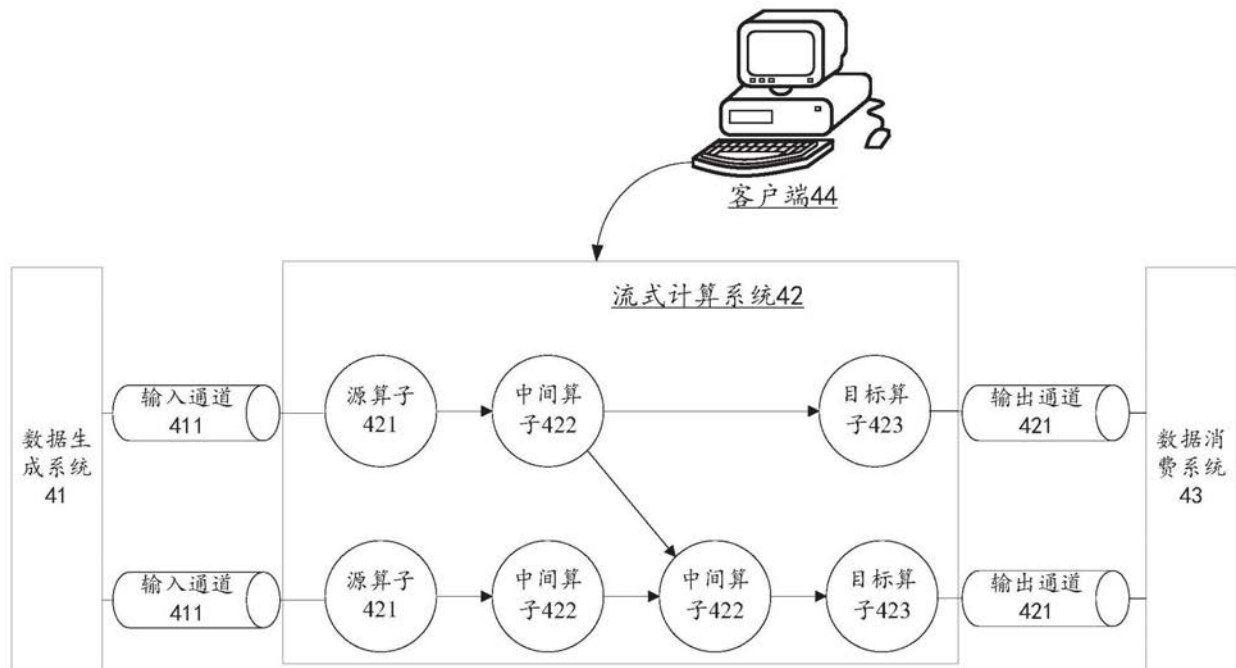


图4

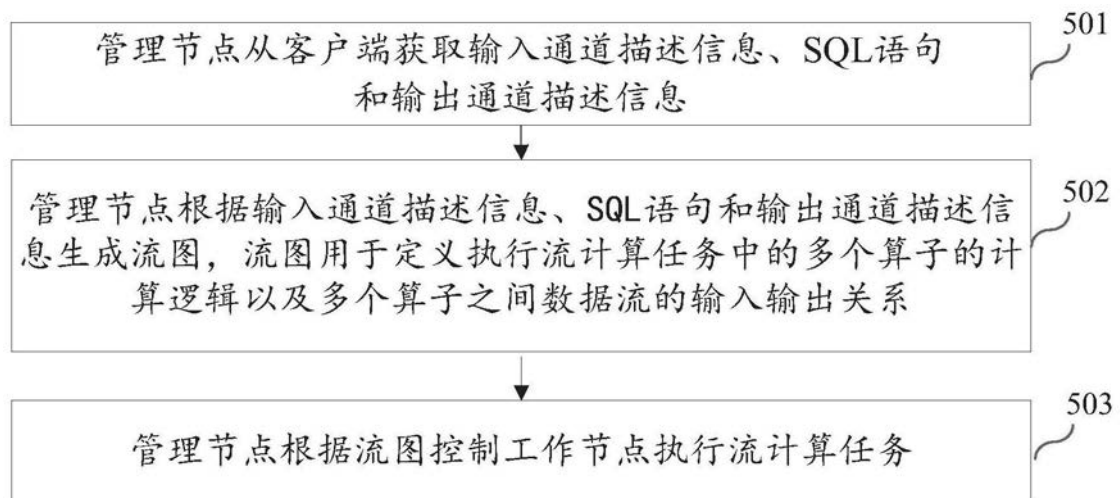


图5

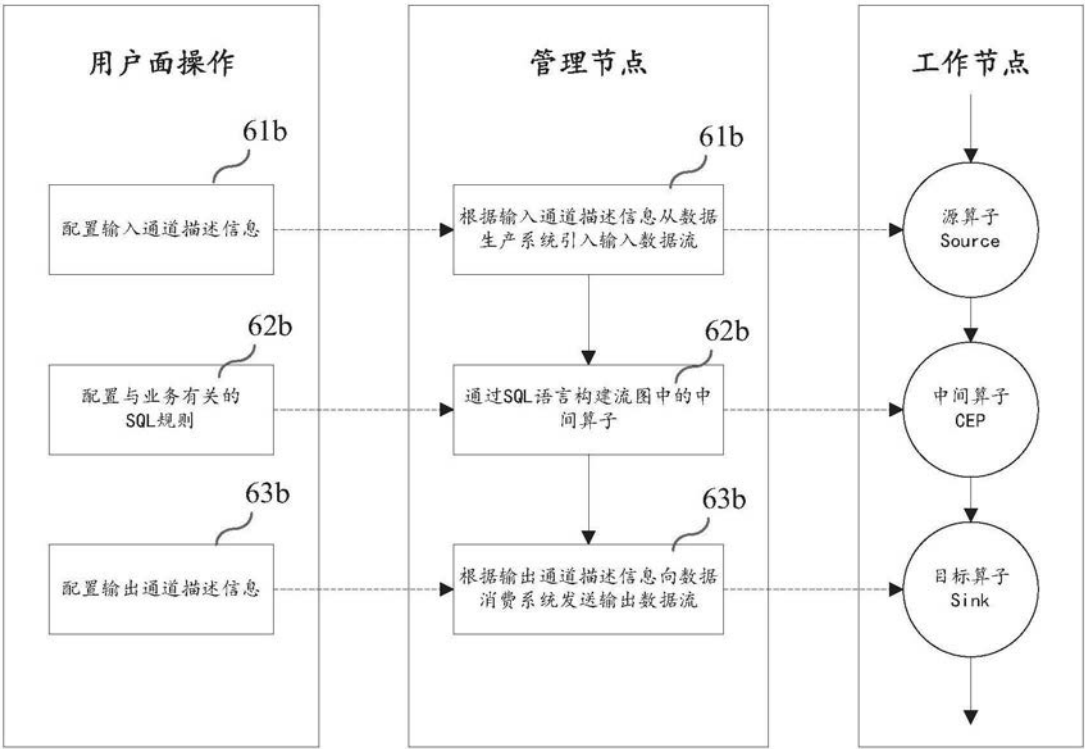


图6

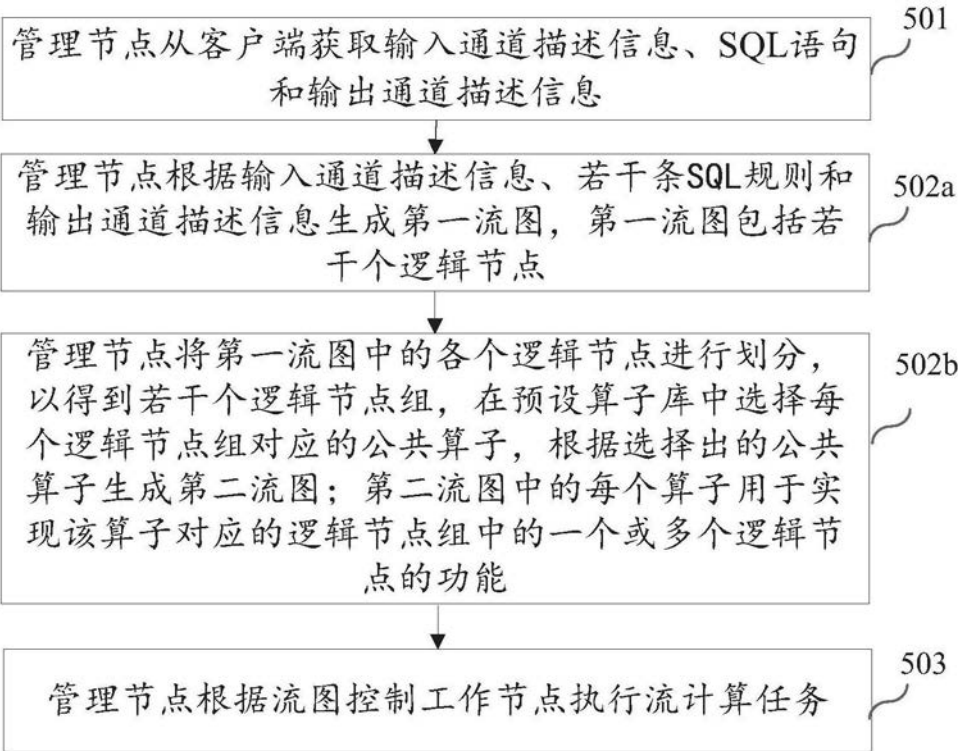


图7

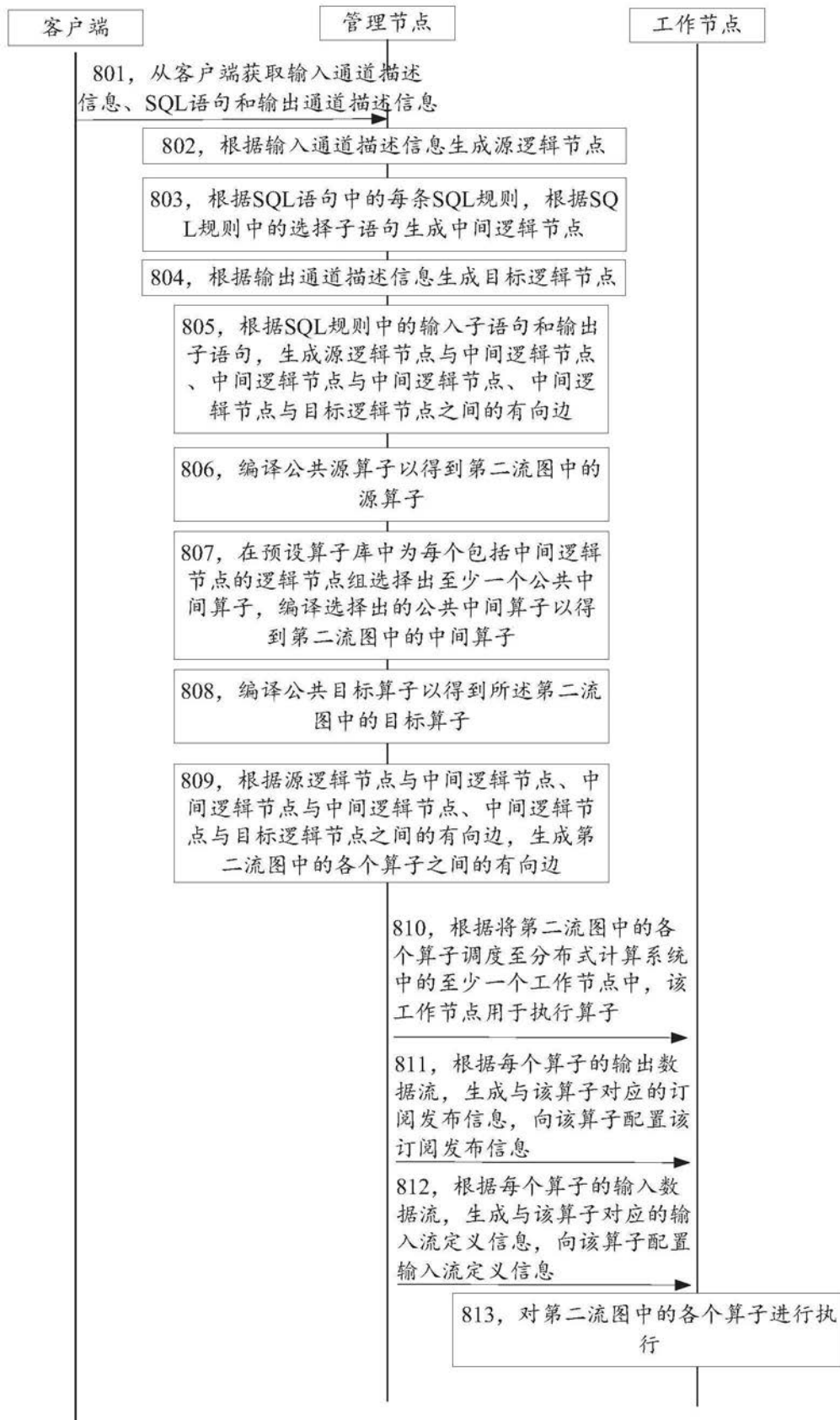


图8A

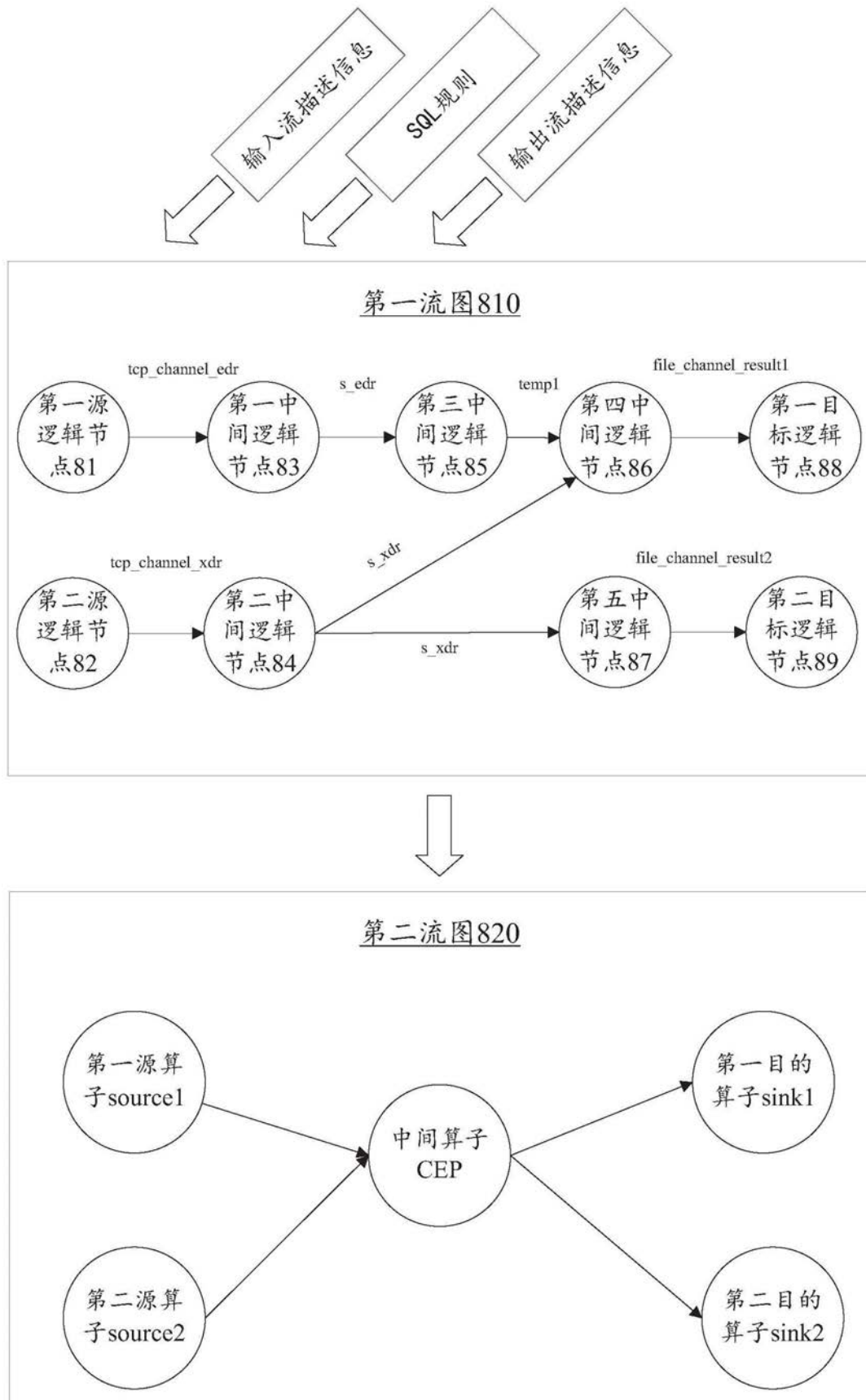


图8B

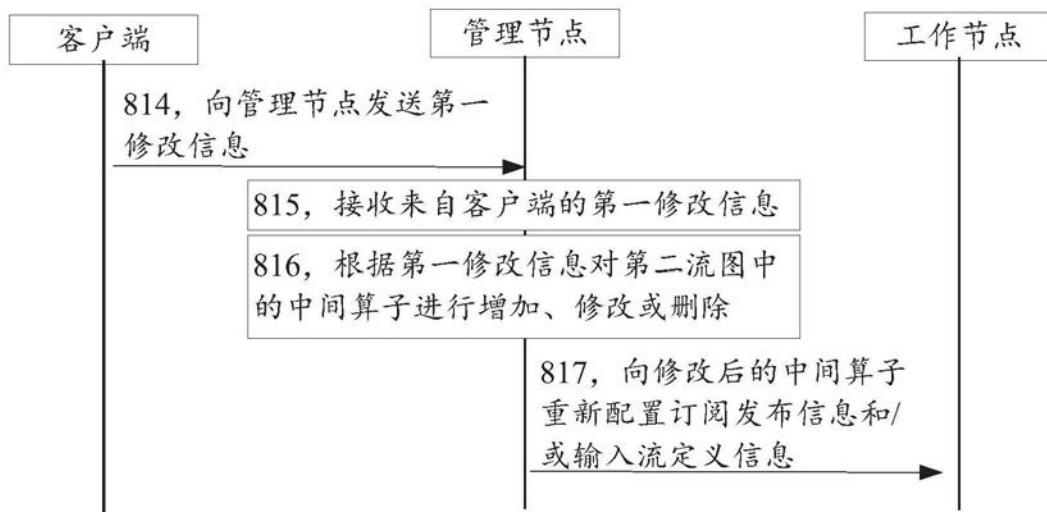


图8C

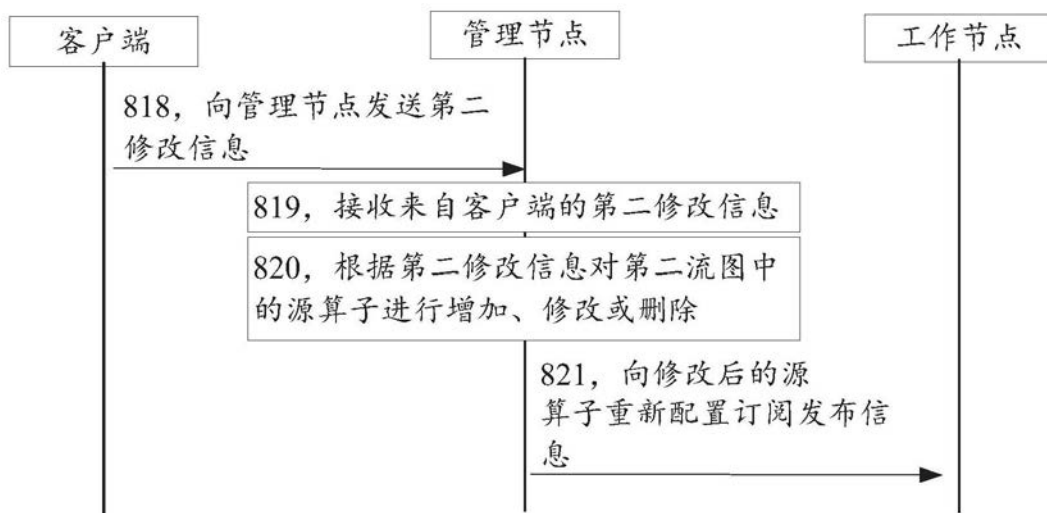


图8D

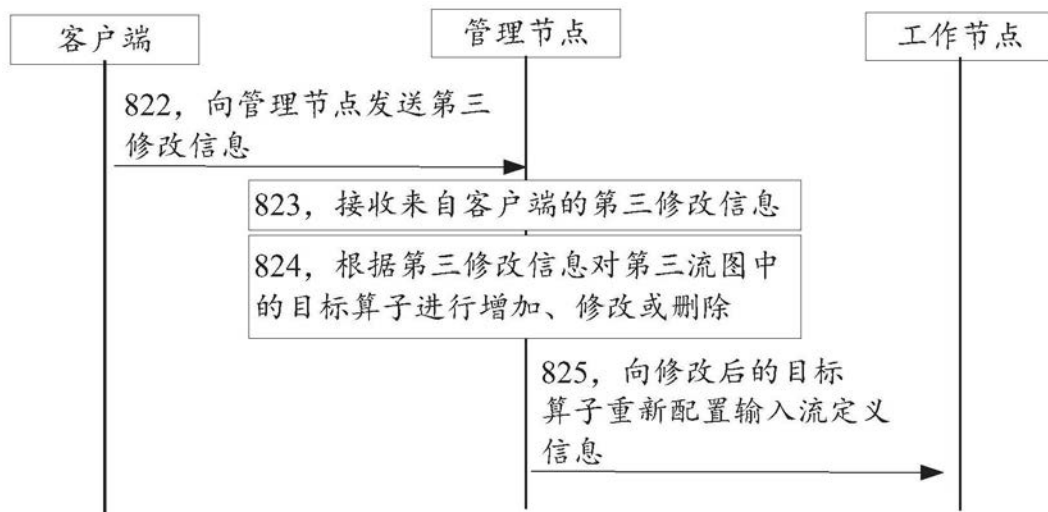


图8E

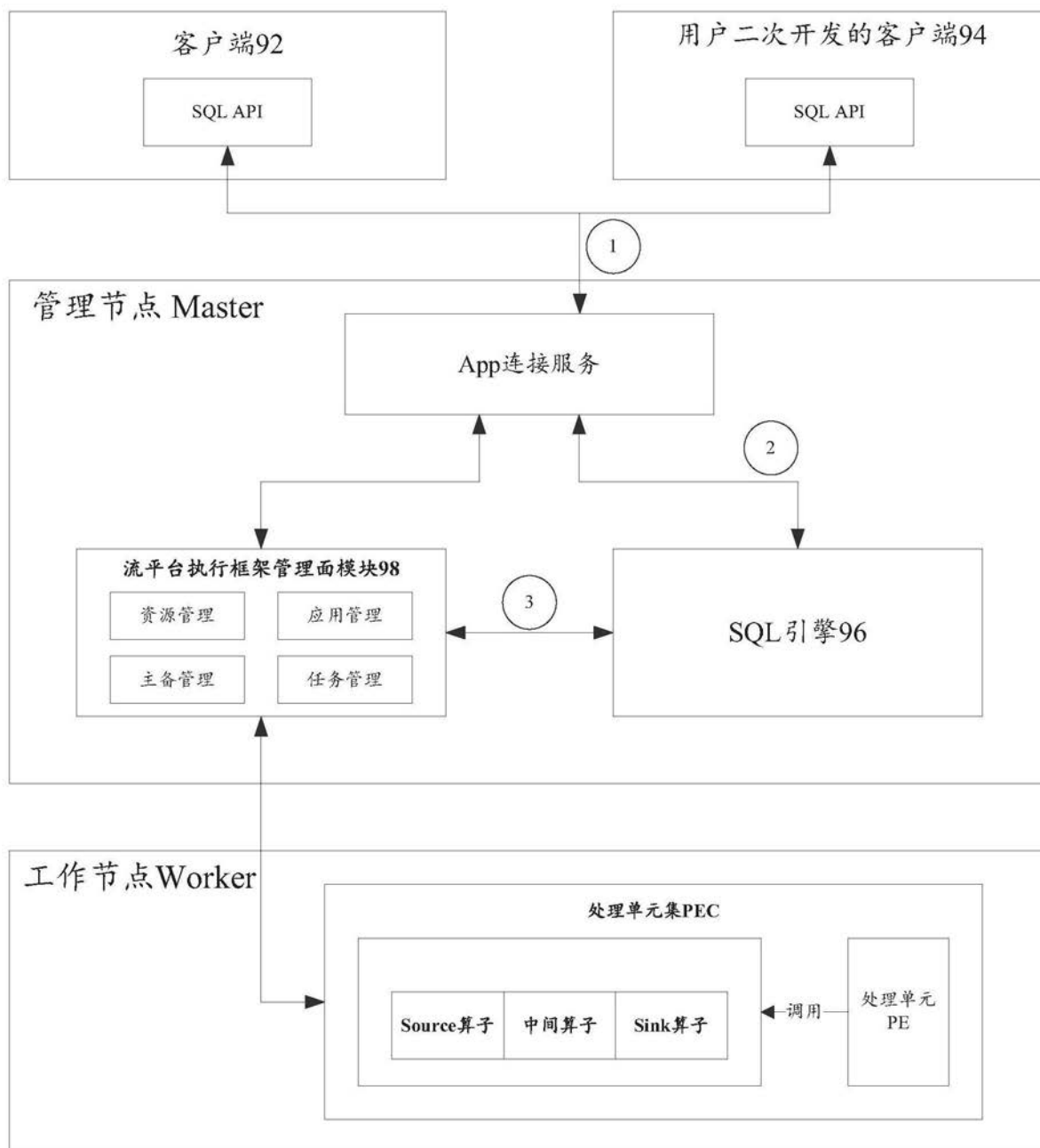


图9A

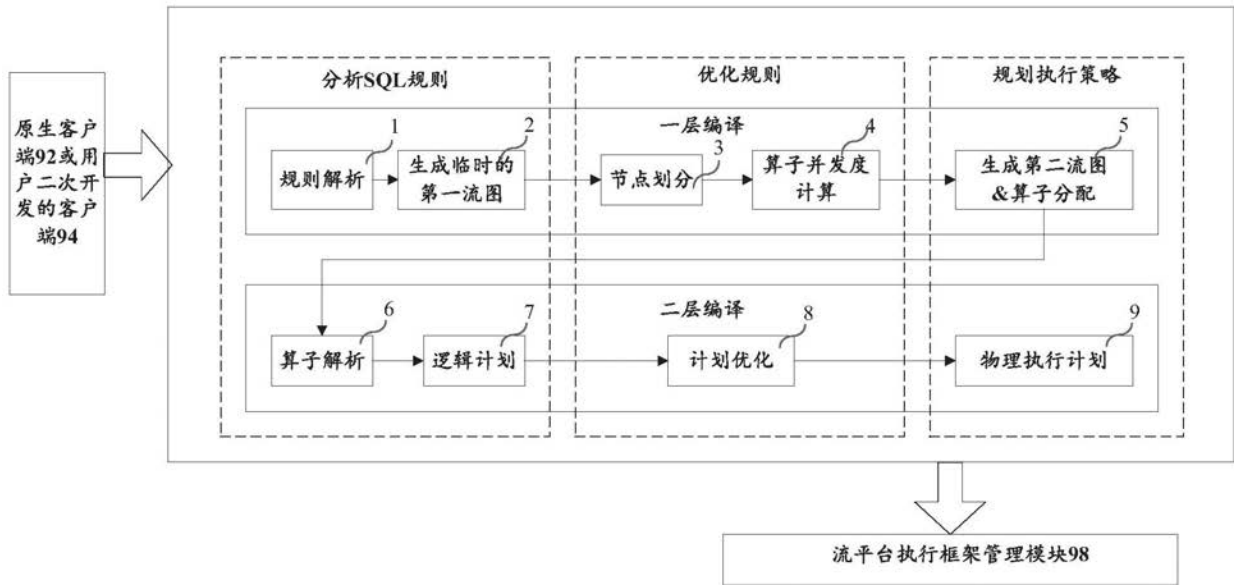


图9B

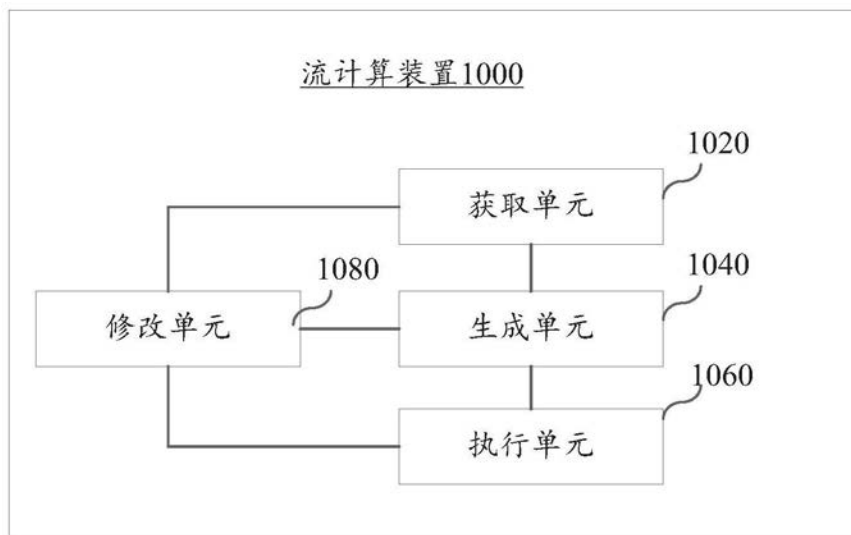


图10

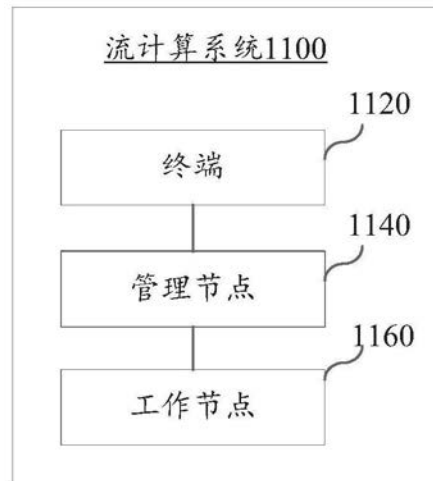


图11